Requests of a Feather Must Flock Together

Anonymous Author(s)

Abstract

Rough proposed page limits after shrinking for the final draft:

- Abstract and Intro: 1.5 pages
- Background: 1 page
- Motivation: 2.5 - 3 pages
- Design and Implementation: 2.5 - 3 pages
- Eval: 3 pages (including micro-benchmarks and ablation studies)
- Conclusion: 0.5 pages

1 Introduction

need to add 1-2 images?

Large language models (LLMs) [13, 26, 35, 36] have been increasingly deployed across various domains, achieving remarkable performance in applications including conversational assistants [11, 14], code generation [16], and retrieval-augmented generation (RAG) [20, 39]. They have started to profoundly impact our work and daily routines. Many cloud companies [5, 15, 25] are racing to provide these applications as hosted services. However, running these applications is very expensive, requiring a large number of hardware accelerators such as GPUs. As deployment grows, the latency and resource efficiency of both online and offline inference have become increasingly critical. In response, a plethora of systems have been deployed to optimize different aspects of LLM serving, such as scheduling strategies [2, 28, 45, 47], parallelization techniques [32, 37], memory management [18, 31, 42], quantization [10, 22], and kernel-level performance [27, 41].

To understand the landscape of optimizations covered in this paper, we briefly review the key technical foundations. LLM inference operates in two distinct phases: a compute-bound **prefill** phase that processes all prompt tokens in parallel to produce the first output token, and a memory-bound **decode** phase that generates one token per iteration. The decode phase relies on a **KV cache** — stored keys and values from prior tokens — which grows with context length and increases the memory footprint.

At the hardware level, GPUs execute thousands of parallel threads across Streaming Multiprocessors (SMs), each with local shared memory and tensor cores, all backed by a shared L2 cache and global memory; the interplay between compute throughput and memory bandwidth at this level shapes the design of nearly every optimization discussed in this paper. this paragraph is not really needed in intro, can be easily skipped.

Mismatched compute profiles of prefill and decode motivate a large body of work on **batching and scheduling**. Systems have explored strategies that either interleave [2], disaggregate [47], or spatially co-schedule [17, 23] the two phases.

Many practical workloads, such as few shot prompting [9, 30], system prompts [1], and RAG [21], also exhibit shared prompt prefixes across requests. Reusing the KV cache of these shared prefixes avoids redundant computation and reduces memory traffic. Systems such as SGLang [44] and vLLM [19] manage this through radix trees and Merkle-tree-inspired hashing schemes, respectively.

A complementary line of kernel-level work [27, 38, 40, 41] pushes this idea further by restructuring how queries are grouped and KV data is loaded within the attention kernel itself, enabling the reuse of shared KV data and reducing redundant global memory traffic at the GPU execution level.

Despite these advances, a precise empirical understanding of *why* and *how* shared prefix structure affects decode throughput has been lacking. In this paper, we present a series of controlled experiments that systematically quantify the relationship between prefix structure and inference performance.

Our central finding is that decode-phase throughput is primarily governed by KV cache memory access patterns rather than raw arithmetic compute. (should we reword the previous statement?) Intuitively, during the decode phase, every new token generated by a request requires attending to *all* previously seen tokens, which means that the GPU must repeatedly read through the entire KV cache at every generation step. When multiple requests in a batch share the same prompt prefix, they traverse the same regions of GPU memory at nearly the same time. This synchronized access improves both *spatial locality* (the hardware prefetcher can reliably anticipate and load upcoming memory regions) and *temporal locality* (a memory block fetched for one request is likely to be reused by a neighbouring request before being evicted). Together, these effects increase L2 cache hit rates and sustain high memory bandwidth utilization.

We validate this hypothesis through several controlled experiments. First, we show that even a single request, out of a large batch, with a different prefix, can reduce decode throughput by nearly 2×. This stark sensitivity arises because the GPU memory access stream becomes interleaved across two disjoint KV cache regions, disrupting prefetcher confidence and reducing temporal cache reuse. Second, we demonstrate that throughput improves smoothly and monotonically as the fraction of shared prefix tokens increases: moving from no sharing to full sharing yields over 60% higher

throughput. Third, we find that the position of sharing within a sequence also matters; since all requests traverse the KV cache in lockstep during decode, sharing at the beginning of a sequence provides the greatest locality benefit. Once requests diverge, synchronized memory access is disrupted, and subsequent sharing in later parts of the sequence provides insignificant returns. Fourth, beyond locality benefits, we show that when the number of simultaneously active prefixes becomes large enough to exhaust GPU memory capacity, the system is forced to evict and recompute KV cache blocks, causing an additional sharp decline in throughput and effective batch size. This highlights that prefix-aware scheduling must account for both memory access locality and GPU memory capacity constraints.

We further characterize the interplay between batch size and prefix structure. Increasing batch size initially improves throughput by better utilizing GPU tensor cores; however, once compute resources are saturated, adding more requests only increases KV cache traffic without proportional throughput gains, making memory bandwidth the bottleneck. Under end-to-end execution, including both prefill and decode phases, we show that prefix-homogeneous batches can outperform larger heterogeneous batches once decode lengths are sufficiently long. This is because each decode step requires another full sweep of the KV cache, allowing the locality benefits of aligned memory access to compound over time. Interestingly, even a homogeneous batch of 50 requests can surpass a heterogeneous batch of 800 requests when decode lengths are long enough, demonstrating that memory locality can outweigh the loss in tensor core utilization.

Finally, we examine the cost of detecting prefix sharing at runtime. Existing systems such as SGLang [44] use radix-tree-based schedulers that perform token-by-token comparisons, dynamic node splitting, and repeated tree traversals to identify shared prefixes. We show that for a workload of 2000 requests with 10k-token prefixes, these scheduling policies incur CPU overhead of 78-89 seconds, which is on the same order as the GPU execution time itself. This reveals a fundamental inefficiency: the very mechanism used to exploit prefix sharing can itself become a bottleneck, motivating a shift towards a lightweight prefix identification scheme.

should the metric come out here?

Motivated by these insights, we propose a scheduling system built around two complementary components. The first is the **Chunked Hash Tree (CHT)**, a lightweight data structure for fast prefix detection. Rather than token-by-token radix tree traversals, CHT represents each request as a compact vector of cumulative prefix hashes computed at fixed token intervals. A min-heap ordered by per-request missing counts - the number of prefix chunks not yet covered by the active batch - enables $O(\log W)$ identification of the best prefix-sharing candidate among W waiting requests. On the top of this, lazy heap updates keep the total CPU overhead minimal, even for thousands of long-prompt requests.

The second component addresses the question of when to stop adding requests to a batch. Growing a batch improves GPU utilization, but each new request with a diverging prefix shrinks the overall shared prefix length and reduces locality. We frame this stop-or-continue decision as a reinforcement learning problem. At each step of batch construction, the scheduler observes the current batch size, the shared prefix length before and after admitting the next candidate, and the number of waiting requests that share the same prefix level (info about all the state variables not needed probably). Tabular policies, trained online from observed decode throughput, learn to balance these competing objectives adaptively across different GPU configurations and workload conditions, which a fixed heuristic policy cannot do reliably. Together, the CHT and the learned batching policy form an end-to-end scheduling system that simultaneously reduces CPU scheduling overhead and improves decode-phase throughput by constructing large, prefix-homogeneous batches.

add eval results once ready!

1.1 Contributions of this paper

In summary, the paper makes the following contributions:

- We conduct an in-depth analysis of the bottlenecks that lead to reduced DRAM bandwidth utilization for prefix-shared workloads.
- We also identify several CPU-level overheads while detecting prefix sharing in a batch of requests.
- We design and implement an efficient data structure that effectively organizes requests and reduces the scheduler's CPU overhead for such workloads.
- We come up with a metric that, in turn, drives our batching decisions. We also employ reinforcement learning policies to guide batch formation.
- Experimental evaluations on real-world workloads show that we outperform state-of-the-art inference engines in both latency and throughput while achieving significantly better DRAM bandwidth utilization.

1.2 Overall paper organization

We present related work in this field in §2. We provide an extensive set of experiments that demonstrate our insights and key ideas in §3. We next present the design and implementation of our ideas in §4. We evaluate our implementation with real workloads against state-of-the-art baselines in §5, along with micro-benchmarks and an ablation study, followed by the conclusion in §6.

2 Background

add delta of each section from our work?

2.1 LLM Inference and Attention

LLM inference operates auto-regressively: each output token is generated from the hidden state of the preceding token.

Every request passes through two phases. In the **prefill** phase, all prompt tokens are processed in parallel to produce the first output token; its latency is called *time-to-first-token* (TTFT). In the **decode** phase, one token is generated per iteration, with per-token latency called *time-between-tokens* (TBT). Prefill is compute-bound due to high token parallelism, while decode is memory-bound due to sequential generation against a growing context.

Each decode step involves three major computations: Self-Attention, Query-Key-Value-Output (QKVO) projection, and a Multi-Layer Perceptron (MLP). Self-attention models global token dependencies as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V} \quad (1)$$

where d is the head dimension. During the decode phase, the query $\mathbf{Q}$ for the current token attends to all prior keys $\mathbf{K}$ and values $\mathbf{V}$. Since $\mathbf{K}$ and $\mathbf{V}$ are reused across steps, they are cached in GPU global memory as the **KV cache**, eliminating redundant recomputation at the cost of an increasing memory footprint. These two characteristics of LLM inference—the asymmetry between the prefill and decode phases and the memory pressure of the KV cache—motivate much of the systems work discussed below. To delve deeper into this, we first explain how GPUs operate and then present a case study of existing work.

need to add time spent in attention and so on?

2.2 GPU Execution Model

what all things to mention? The GPU's hardware is arranged in a hierarchy that supports execution at a scale of hundreds of thousands of parallel threads. The main processing unit of a GPU is a Streaming Multiprocessor (SM), with modern GPUs containing around a hundred SMs. Each SM has an L1 cache and shared memory, along with tensor cores for accelerated general matrix multiplication (GEMM) and execution units for integer and floating point operations. The shared memory is a user-addressable partition of the L1 cache. The GPU memory is accessed by SMs through the shared L2 cache. mention about prefetcher here, and cite something? Is there some abrupt break in the flow just after this?

2.3 Batching and Scheduling for LLM Inference

A central challenge in LLM serving is that prefill and decode iterations have different compute profiles, making it difficult to mix them in a batch without interference. vLLM [19] laid the groundwork for flexible batching by introducing *PagedAttention*, which stores the KV cache in non-contiguous (reviewers might target us on this, basically if they are physically non-contiguous, then how are the benefits coming in? we need to clarify that initial allocations all contiguous.) fixed-size memory blocks analogous to virtual memory paging, eliminating fragmentation and enabling KV blocks to be shared across requests. Building on this, Sarathi Serve [2] addresses the throughput-latency tradeoff more directly by splitting large prompts into fixed-size chunks via *chunked prefills* and using *stall-free batching* to coalesce these chunks with ongoing decode operations, eliminating the generation stalls caused by long prefill iterations blocking active decodes. Where Sarathi-Serve mitigates interference by interleaving, DistServe [47] eliminates it entirely by disaggregating prefill and decode into separate GPUs, independently optimizing resource allocation and parallelism for each phase. POD-Attention [17] takes a different approach, keeping both phases on the same GPU while introducing a fused attention kernel with SM-aware CTA scheduling to overlap their compute and memory-bound operations concurrently on the same multiprocessor. Similarly, Bullet [23] exploits spatial-temporal GPU resource sharing between prefill and decode, coupling an SLO-aware scheduler with a dynamic SM resource manager to balance throughput and latency objectives without physical separation. bullet is one of the latest papers, so nice to describe, but not too much relevant contribution?

Beyond phase co-scheduling, several systems focus on request-level batching policies to improve efficiency. BucketServe [45] reduces padding waste by grouping requests with similar sequence lengths into buckets, improving GPU memory efficiency. Pang et al. [29] takes a runtime-adaptive approach, dynamically adjusting batch size based on live GPU memory usage and latency SLA constraints. For offline settings, BlendServe [43] introduces a resource-aware prefix tree and a dual-scanner scheduling algorithm that reorders and co-schedules requests with complementary compute and memory demands to maximize GPU utilization. Collectively, these works highlight that batching and scheduling decisions, whether at the phase, request, or resource level, have a decent impact on overall inference efficiency.

2.4 Prefix Sharing in LLM Serving

Many real-world workloads, such as few-shot prompting, system prompts, and retrieval-augmented generation (cite for each category, or if done in the intro and not needed here) involve requests that share a common prompt prefix. Reusing the KV cache of shared prefixes avoids redundant computation and reduces memory pressure, but doing so efficiently requires careful coordination between the serving system and memory management. SGLang [44] was among the first to formalize this, introducing RadixAttention, which uses a radix tree to identify and reuse KV caches across requests with shared prefixes in a multi-call serving setting. Radix trees explicitly store token sequences in a hierarchical structure where common prefixes share the same path. We show a simple example of a radix tree in Figure 1.

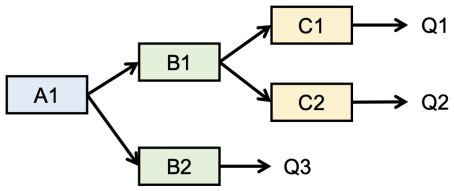


Figure 1. An example of a radix tree from [27], where queries Q1, Q2 share A1, B1 sub-sequences with each other, and only A1 with query Q3.

vLLM [19] also implements automatic prefix caching [33] using a hashing scheme inspired by Merkle trees, as shown in Figure 2. Each KV-cache block is assigned a hash that depends on both the tokens inside the block and the hash of the preceding prefix block. This effectively forms a chain of hashes representing the entire prefix. If two requests produce the same sequence of hashes, the system can freely reuse the corresponding KV-cache blocks.

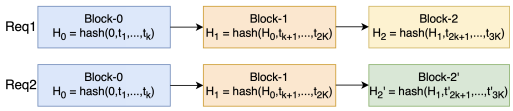


Figure 2. An example of vllm Merkle Tree.

BatchLLM [46] extends the idea of prefix sharing to large offline batches, explicitly grouping requests by shared prefixes for global KV cache reuse, then reordering them and applying memory-aware token batching to improve the mix of prefill and decode tokens. BlendServe [43] similarly leverages a resource-aware prefix tree, but pairs it with a scheduling algorithm that jointly optimizes for both prefix reuse and complementary compute-memory demand balancing. Together, these systems establish that effective prefix sharing requires not just identifying common prefixes but also carefully scheduling the order in which requests are processed to maximize KV cache hit rates and memory efficiency.

2.5 Prefix-Aware Attention Kernels

While the systems above handle prefix sharing at the serving and scheduling level, a complementary line of work focuses on exploiting shared prefix structure directly inside the attention kernel, reducing redundant global memory traffic at the GPU execution level. PAT [41] takes a direct approach by packing queries that share the same prefix into a single CTA, so the shared KV cache is loaded from global memory once and broadcast across all grouped queries. It introduces a pack-forward-merge pipeline with a prefix-aware pack scheduler, multi-tile attention kernels, and multi-stream execution to improve hardware utilization. DEFT [38] addresses the same problem in tree-structured decoding by grouping queries according to their shared KV prefixes (KV-guided grouping), ensuring each prefix segment is fetched exactly once; it further introduces Flattened Tree KV Splitting to balance KV partitions across thread blocks for uniform GPU utilization. ChunkAttention [40] approaches the problem from a memory layout perspective, storing KV caches in a prefix tree of chunked KV tensors and introducing a two-phase partition kernel, chunk-first and sequence-first, to improve memory locality and parallelism when operating over shared prefix chunks. (improve chunk attention description) FastTree [27] targets radix-tree-structured workloads specifically, grouping queries that share the same tree context to reuse KV cache directly from shared memory, reducing global memory pressure and better utilizing tensor cores. Across all these works, the common insight is that the key to efficient prefix-aware attention lies in restructuring how queries are grouped and how KV data is loaded.

3 Motivation

3.1 Experiment Setup and Overview

A common assumption in LLM serving is that larger batches are strictly better—more requests per batch means higher GPU utilization and greater throughput. Our central claim challenges this assumption: **a moderately sized batch of prefix-homogeneous requests can substantially outperform a larger batch of prefix-heterogeneous requests during the decode phase.**

The key insight is that decode-phase throughput is largely determined by the memory access pattern of the KV cache. When all requests in a batch share a common prefix, their attention operations traverse the same KV cache regions at roughly the same time, improving spatial and temporal locality in the GPU memory hierarchy. In contrast, when requests have heterogeneous prefixes, KV accesses become interleaved across disjoint memory regions, disrupting locality and reducing effective bandwidth utilization—a cost that cannot be offset simply by increasing batch size. Importantly, we demonstrate that these locality gains can be achieved without any modifications to the attention kernels themselves, making the approach directly applicable to existing inference stacks.

To validate this, we design a series of controlled experiments that systematically vary (i) the amount of prefix sharing within a batch, (ii) the number of distinct prefix groups, and (iii) the batch size and decode length. Across these experiments, we measure decode throughput and memory behaviour to understand how prefix structure affects inference performance. Each experiment concludes with a takeaway summarizing the key insight.

All experiments use the LLama3-8B [3] model. Experiments are performed on an RTX6000 Ada Generation GPU [24], equipped with 48GB of GDDR6 memory and a 96MB

L2 cache. We use the vLLM [19] inference engine with a default maximum batch size of 500 unless stated otherwise. We primarily focus on decode-phase throughput, measured in output tokens per second. To isolate decode behaviour, we perform a warm-up run such that all prefix KV caches are resident in GPU memory prior to measurement. This ensures that the per-token compute cost remains constant across experimental settings and that observed differences arise primarily from memory behaviour and batching effects. Later, we also compare end-to-end throughput, which includes the time taken to prefill the queries. For this model, a single 10k-token request generates approximately 1.4GB of KV cache data (across all layers, for both keys and values). Within each layer, KV tensors corresponding to consecutive tokens are stored contiguously in memory. We pick requests from the L-Eval [4] dataset, and we prune the size of each request as required by the experiment. We use the FlashAttention [12] backend throughout all the experiments.

3.2 Locality and Sharing

We begin by examining how *prefix heterogeneity* within a batch affects decode throughput. Prefix heterogeneity refers to the presence of requests with different input prefixes in the same batch. When requests share the same prefix, they access the same KV cache regions during decode; when prefixes differ, the KV accesses correspond to different memory regions.

To quantify the impact of prefix heterogeneity, we design a controlled experiment with two large system prompts, A and B, each of length 10,000 tokens. We form a batch of 500 requests, where each request consists of either prefix A or B, followed by a unique suffix of length 20 tokens (Figure 3). Each request then generates 50 decode tokens. Let $f \in [0, 1]$ denote the fraction of requests using prefix A. When $f = 0$ or $f = 1$, the batch is homogeneous, and all 500 requests share a single prefix. When $0 < f < 1$, two distinct prefix families coexist within the same batch. We do a warmup run before measuring the decode throughput, which ensures that the KV caches corresponding to both the prefix groups are already resident in the GPU memory, and the amount of compute involved is the same across all values of f.

Figure 4 shows decode throughput as a function of f. Throughput is maximized when the batch is homogeneous ($f = 0$ or $f = 1$). However, introducing even a single request from the other prefix family causes a sharp degradation. Specifically, at $f = 0.002$ (one request out of 500), throughput drops by nearly 2×. This demonstrates that decode throughput is extremely sensitive to even minimal prefix heterogeneity within a small batch.

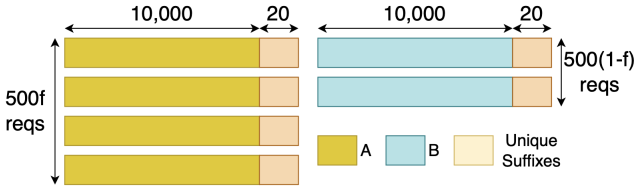


Figure 3. Visualization of Heterogeneous Batches

Do we need to mark fractions such as 0.002 or 0.01 more clearly in this graph?

To better understand this effect, we measure mean DRAM activity, defined as the fraction of cycles during which the DRAM pipe is active. As shown in Figure 5, homogeneous batches exhibit over 50% higher DRAM activity compared to mixed-prefix settings.

A 10k-token prefix corresponds to over 1GB of KV cache data (across all layers). Even with sharing across requests, the active KV working set during decode substantially exceeds the 96MB L2 cache capacity of the GPU, particularly after accounting for model weights. Thus, decode is fundamentally memory-bound, and only a small portion of the KV cache can reside in L2 at any given time.

present max bandwidth numbers here

During each decode step, every request attends to its entire prefix. Because the prefix length (10k tokens) dominates both the decode length (50 tokens) and the query length, memory accesses are largely determined by prefix traversal. In the homogeneous case, all requests sweep through the same KV blocks nearly simultaneously. This produces two beneficial effects: (i) **Spatial Locality:** Sequential traversal of contiguous KV tensors allows the hardware prefetcher to reliably fetch upcoming cache lines from DRAM into L2; (ii) **Temporal Locality:** Since requests access the same KV regions at roughly the same time, a cache line fetched for one request is likely reused by neighbouring requests before eviction. Together, these effects increase L2 hit rates and sustain high DRAM pipeline utilization, resulting in higher throughput. Since we use a query-centric (FlashAttention) backend, most SMs process queries from the dominant prefix group even when $f = 0.002$. Therefore, we attribute the performance gains primarily to improved L2 cache reuse rather than increased L1 cache locality.

In contrast, when even a small fraction of requests uses a different large prefix, the KV access stream becomes interleaved across two disjoint memory regions. This disrupts prefetcher confidence and weakens spatial locality. More importantly, temporal locality collapses; cache lines fetched for prefix A are unlikely to be reused by prefix B requests before eviction. The result is reduced effective bandwidth utilization and significantly lower throughput.

Takeaway 1: Decode throughput is highest when all requests in a batch share the same prefix, allowing their attention operations to access the same KV cache regions at

nearly the same time. Even a single request with a different prefix forces the GPU to interleave accesses across multiple KV regions, breaking cache locality and sharply reducing effective memory bandwidth.

The previous experiment raises a natural follow-up question: how does throughput change when requests partially share their prefixes rather than being completely identical or completely different? Our hypothesis is that if decode performance is primarily determined by KV cache locality, then increasing the amount of shared prefix across requests should progressively improve throughput. Larger shared regions should allow more requests to access the same KV blocks within a short time window, strengthening cache reuse and improving effective memory bandwidth utilization.

To test this hypothesis, we isolate the effect of prefix sharing by constructing a batch of 100 requests, each with a total length of 2000 tokens. In this experiment, every request shares a common prefix of length $2000 \times f$ tokens, where $f \in [0, 1]$. The remaining $2000 \times (1 - f)$ tokens are unique to each request, as illustrated in Figure 6. As before, a warm-up run ensures that all the 2000-tokens of each of the 100 requests are already present in the GPU memory, so that we can attribute the trend in throughput solely to the locality present amongst the requests.

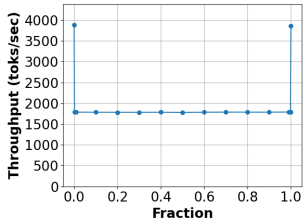


Figure 4. Throughput vs Fraction of A prefix requests

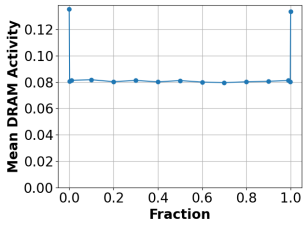


Figure 5. Mean DRAM Activity vs Fraction of A prefix requests

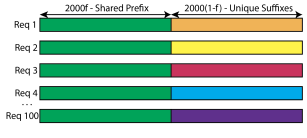


Figure 6. Visualization of $2000 \times f$ tokens shared by requests

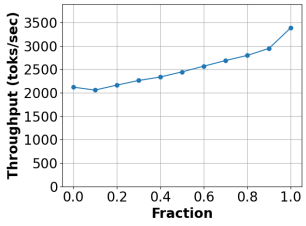


Figure 7. Throughput vs Fraction of Sharing

As shown in Figure 7, there is a smooth increase in throughput with f. When there is no shared prefix ($f = 0$), throughput is at its lowest. As sharing increases, throughput improves consistently. When all requests share the entire prefix ($f = 1$), throughput is more than 60% higher than in the no-sharing case. As f increases, a larger portion of the KV cache becomes common across the batch. Since decode for each token repeatedly sweeps the entire KV cache, the shared region is accessed by multiple requests within a short time window, improving contiguity across memory accesses. Both temporal and spatial locality, therefore, improve with f, resulting in improved throughput.

Takeaway 2: Decode throughput increases smoothly with the amount of shared prefix across requests. Greater sharing enlarges the KV regions accessed jointly by the batch, strengthening cache reuse and improving effective memory bandwidth utilization.

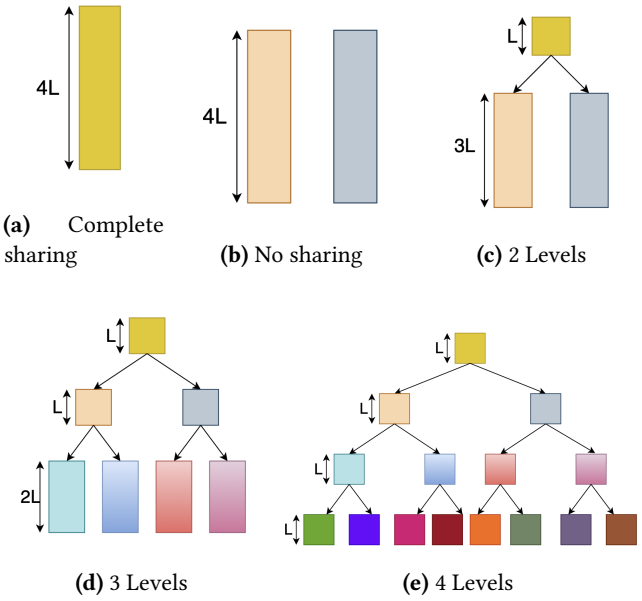
The previous experiment varies only the amount of sharing while assuming that the shared tokens appear at the beginning of the sequence. This raises a natural question: does the position of sharing within a sequence also affect decode throughput? Our hypothesis is that because decode proceeds token by token and all requests traverse their prefixes at the same time, sharing that occurs earlier in the sequence should have a larger impact on locality than sharing that occurs later. Early shared segments allow the batch to access the same KV regions simultaneously, while later divergence breaks this alignment. To test this hypothesis, we design five sharing patterns: complete sharing, no sharing, and sharing with 2, 3, or 4 levels, as illustrated in Figure 8. In all configurations, the total sequence length of each request is fixed at $4L$, and only the branching structure differs. We measure decode throughput across these five settings for three values of L, as shown in Figure 9.

The complete sharing pattern consistently achieves substantially higher throughput than all others. This behavior is expected: full sharing preserves synchronized traversal of the KV cache throughout the entire sequence, maximizing both temporal and spatial locality.

A more subtle observation is that configurations with 2, 3, and 4 levels exhibit similar throughput across different values of L, despite differing branching patterns. This can be explained by the timing of memory accesses during decode. Requests within a batch issue memory accesses roughly simultaneously. During the initial shared segment of length L, all configurations behave identically; every request accesses the same KV region, fully exploiting locality. Once branching occurs, the batch becomes heterogeneous, and locality degrades. As established earlier, the mere presence of heterogeneity within a batch is sufficient to disrupt cache reuse sharply, the marginal cost of introducing additional distinct prefix groups beyond the first is comparatively small, since synchronized traversal has already been broken. Whether the batch ultimately splits into two, four, or eight groups in later segments therefore has only a limited additional impact. Moreover, as expected, the no-sharing pattern achieves the lowest throughput of all.

Takeaway 3: Because decode-phase memory accesses from all sequences occur roughly together, the amount of

sharing at the beginning of the sequence is the primary determinant of locality and throughput.



(a) Complete sharing **(b)** No sharing **(c)** 2 Levels

(d) 3 Levels **(e)** 4 Levels

Figure 8. Visualization for different sharing patterns

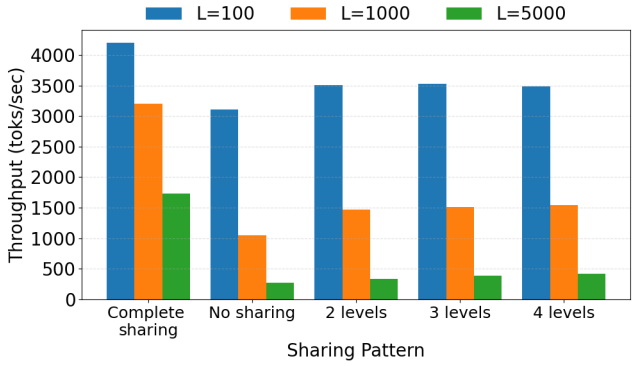


Figure 9. Throughput vs sharing patterns for different sequence lengths

The previous experiments examine how prefix sharing affects decode throughput when the KV caches of all active requests fit within GPU memory. However, in some practical serving workloads, a system may need to handle many distinct prefixes simultaneously. This raises another important question: what happens when the number of distinct prefixes becomes large enough that their KV caches cannot all reside in GPU memory? Our hypothesis is that when the number of active prefixes grows beyond the available GPU memory capacity, the system will be forced to evict KV cache blocks. These evictions require recomputation when the corresponding requests resume their decode phases, leading to reduced throughput and smaller effective batch sizes.

To study this effect, we construct an experiment with N different shared prefixes $P_1, P_2 \ldots P_N$, each of length 5000 tokens. We create batches such that an equal number of requests corresponds to each prefix. Figure 10 shows the resulting throughput and average batch size as N increases. As observed earlier, throughput drops when moving from $N = 1$ to $N = 2$, primarily due to reduced temporal and spatial locality. From $N = 2$ to $N = 20$, throughput remains relatively stable, indicating that locality degradation alone does not worsen performance beyond the initial heterogeneity. However, there is a sharp decline in throughput from $N = 20$ to $N = 50$. This drop occurs because the available GPU memory is no longer sufficient to hold the KV caches for all active prefixes simultaneously. This leads to frequent KV cache evictions and recomputations. The effect is also reflected in the reduced average batch size for $N = 50$ and $N = 100$, as the system cannot admit new requests due to limited GPU memory capacity. This phenomenon has also been explored in prior work [28, 46], which highlights the impact of KV cache capacity constraints on inference throughput.

Takeaway 4: Beyond cache locality, the number of simultaneously active prefixes also affects performance through GPU memory capacity limits. Prefix-aware scheduling can mitigate KV cache evictions and improve throughput.

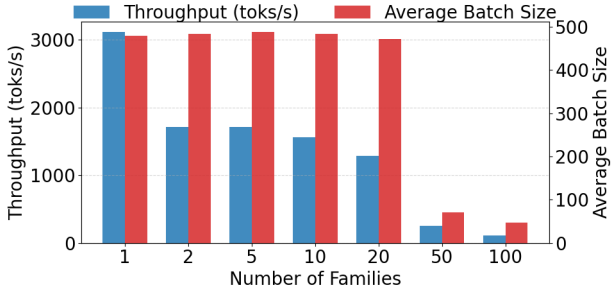


Figure 10. Throughput and Average Batch Size vs Number of Prefix Groups

3.3 Effect of Batch Size on Decode Throughput

The previous experiments show that decode throughput is strongly influenced by prefix structure, which determines how effectively requests can reuse KV cache data. However, prefix sharing is only one factor that affects performance. Another important parameter in LLM serving systems is the batch size, which determines how much parallel work is issued to the GPU during each decode step.

Our hypothesis is that increasing batch size should initially improve throughput by better utilizing GPU compute units, particularly tensor cores. However, once compute resources

are saturated, decode performance should become limited by memory bandwidth, since each additional request increases KV cache traffic.

To study this effect, we evaluate decode phase throughput as a function of batch size. In this experiment, all the requests start from one of the 5 large prefixes of length 2000 tokens each. As shown in Figure 11, throughput increases with batch size due to improved tensor core utilization (Figure 41). However, another point worth noting is that beyond a certain batch size, once the GPU's compute units are already highly utilized, the throughput curve largely flattens. The mean SM Occupancy is shown in Figure 54 (currently linked to appendix plot, can remove this).

Takeaway 5: We should aim to increase the batch size to better utilize GPU compute, but only up to the point when the tensor core utilization saturates, after which decode performance is no longer limited by arithmetic throughput but by KV cache movement between HBM and on-chip caches. At that stage, simply adding more requests to the batch does not yield proportional gains because memory becomes the bottleneck.

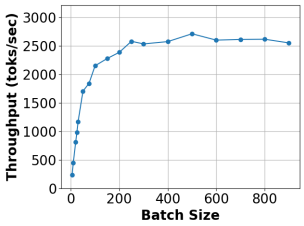


Figure 11. Throughput vs Batch Size

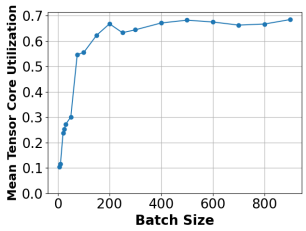


Figure 12. Tensor Core Utilization vs Batch Size

The previous experiment examined how batch size affects decode throughput under a fixed prefix structure. In practice, however, LLM serving workloads involve both prefill and decode phases. These two stages stress different system resources: prefill is primarily compute-intensive, while decode is largely memory-bound due to repeated traversal of the KV cache. Consequently, the benefits of prefix locality observed in earlier experiments may manifest differently when considering end-to-end execution.

Our first hypothesis is that the benefits of prefix locality should become increasingly pronounced as the decode length grows. Prefill is largely a write-heavy workload that constructs the KV cache once, while decode repeatedly reads the entire prefix at every generation step. As the number of decode tokens increases, these repeated KV sweeps amplify the impact of memory locality.

Moreover, there exists a trade-off between large heterogeneous batches and smaller prefix-homogeneous batches. Large heterogeneous batches maximize tensor core utilization, while homogeneous batches preserve aligned traversal of the KV cache during decode.

To evaluate these hypotheses, we measure end-to-end throughput, including both prefill and decode phases. Unlike previous decode-only experiments, we do not perform a warmup run; thus, prefix computation and KV cache construction are included in the reported throughput. We consider a workload consisting of eight distinct prefix groups, each containing 100 requests (800 total). We evaluate several batching strategies: (i) a single heterogeneous batch of 800 requests spanning all eight families; (ii) two batches of 400 requests covering four families each; (iii) four batches of 200 requests covering two families each; (iv) eight batches of 100 requests, each homogeneous; (v) sixteen batches of 50 requests, each homogeneous; and (vi) thirty-two batches of 25 requests, each homogeneous. We further vary the number of decode tokens to study how the prefill–decode balance influences performance.

Figure 13 shows throughput across these configurations. When the decode length is small, throughput is largely influenced by prefill computation. In this regime, larger heterogeneous batches tend to perform well because they maximize tensor core utilization and arithmetic intensity during the prefix processing stage. As we progressively split the large 1×800 batch into 2×400 and 4×200 configurations, throughput decreases slightly, reflecting reduced compute efficiency from smaller batch sizes. In this setting, the benefits of memory locality are not yet dominant enough to compensate for the loss in arithmetic utilization.

However, as we increase the number of decode tokens, there is a shift in the performance trend. Decode requires each request to repeatedly sweep its entire KV cache. When batches are heterogeneous, these sweeps interleave across disjoint memory regions, weakening spatial and temporal locality. As a result, memory accesses become fragmented and effective bandwidth utilization decreases. In contrast, once we transition to fully homogeneous batches (8×100), throughput increases significantly, even though each batch is smaller than the earlier configurations. This improvement arises because all requests within a batch now share the same prefix, restoring synchronized traversal of the KV cache and strengthening both spatial and temporal locality.

Interestingly, even the 16×50 configuration, which further reduces batch size but remains homogeneous, continues to outperform the single large heterogeneous batch (1×800) when the decode length is sufficiently large. This demonstrates that the locality benefits from prefix alignment can outweigh the loss in tensor core utilization. Only when batches become too small (32×25) does throughput begin to decline again. At that point, reduced arithmetic intensity and lower tensor core occupancy dominate, and the compute under-utilization can no longer be compensated by improved memory alignment.

The impact of decode length further illustrates this trade-off. When the number of decode tokens is small (e.g., 10 tokens), a homogeneous batch of size 50 does not outperform a

heterogeneous batch of size 800, since the workload remains largely dominated by prefill computation and compute efficiency plays a larger role. However, when we increase the decode length to 100 tokens, the homogeneous 50-request batch begins to surpass the large heterogeneous batch. This occurs because each additional decode step requires another full sweep of the KV cache, allowing the benefits of aligned memory access patterns to accumulate over time. As decode length increases, execution becomes increasingly memory-bound, and the advantage of prefix-homogeneous batching becomes more pronounced.

Overall, these results highlight a fundamental balance between compute utilization and memory locality. Large heterogeneous batches maximize tensor core usage but fragment KV access patterns. Moderately sized homogeneous batches preserve aligned traversal of the KV cache and can achieve higher effective throughput when decode dominates execution. The optimal operating point lies where compute resources remain well utilized while memory access patterns remain synchronized across requests.

Takeaway 6: Under end-to-end execution, it is often beneficial to process requests in prefix-homogeneous batches, even if this slightly reduces batch size. As the number of decode tokens increases, repeated KV cache sweeps amplify the benefits of memory locality, allowing moderately sized homogeneous batches to outperform larger heterogeneous batches. However, this benefit holds only up to a point: when batches become too small, reduced tensor core utilization and lower arithmetic intensity lead to underutilized compute, causing throughput to decline.

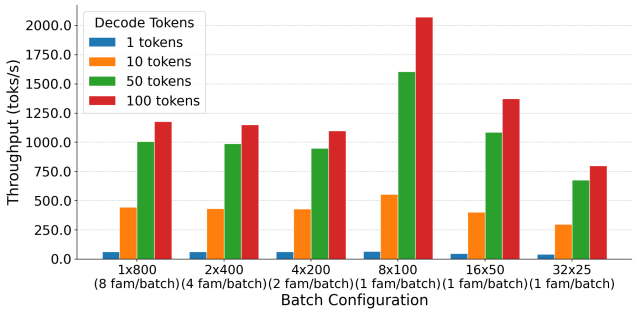


Figure 13. Throughput vs Batch Configurations for varying number of decode tokens

3.4 Overhead of dynamic prefix detection

So far, we have analyzed how the prefix sharing structure and batch size affect the overall inference throughput. Next, we plan to study how the state-of-the-art inference engines identify prefix sharing and what the overall cost involved in doing this is. Although existing inference engines such as SGLang [44] and BatchLLM [46] do not consider making the batch prefix-homogeneous, they still attempt to detect

Policy	Batch Time (s)	Exec (s)	Throughput	TC Util.
vLLM FCFS	3e-6	229.36	436.25	0.67
vLLM User IDs	0.03	105.43	948.54	0.336
vLLM RadixTree	62.95	139.33	717.35	0.20
SGLANG DFS-W	92.15	177.78	562.61	0.22
SGLANG LPM	46.62	149.27	669.92	0.26

Table 1. Comparison of scheduling policies

prefix sharing using radix trees to schedule requests in an order that minimizes KV cache recomputations. In SGLang, two radix-tree-aware scheduling policies are proposed. The first, Longest Prefix Match (LPM), prioritizes requests whose input prefixes match the longest cached prefixes a better definition needed here?. The second, DFS-Weight, performs a depth-first traversal of the radix tree and prioritizes branches that have more active requests. To understand the overhead of these scheduling policies, we implemented a radix-tree-based scheduler inside vLLM and measured its CPU cost. We constructed a workload of 2000 requests, each with a 10k-token prefix, and compared the CPU time spent on scheduling against the total execution time. We also compared these results with an idealized policy in which the user directly provides prefix group identifiers, eliminating the need for any dynamic prefix discovery. describe prefix aware attention here, after reading through, and check if it matches on what we have implemented

Table 1 shows the results. SGLang's LPM and DFS-Weight policies incur 78.24 and 89.06 seconds of CPU time, respectively, on the same order as the GPU computation. Our radix-tree implementation in vLLM reduces this overhead somewhat (57.53 seconds), but the cost remains substantial. In contrast, when prefix group identifiers are provided explicitly, the CPU overhead drops to just 0.03 seconds.

There is a structural reason for this gap: radix trees require token-by-token comparisons during insertion, dynamic node splitting when partial matches occur, and the maintenance of shared paths as requests arrive and complete. Scheduling decisions further require repeated traversals of the tree. With long prompts and high concurrency, these operations accumulate significant constant-factor overhead.

Takeaway 7: Existing approaches introduce significant CPU overhead while detecting prefix sharing, which is at times even comparable to the GPU execution time.

add prefix aware attention tree after reading the paper and implementing their idea

remove vllm radix tree as it has our own optimizations, or we can keep it still?

batchLLM avoids this cost by shifting to offline serving, similar to blendserve.

4 Design and Implementation

As discussed in §3, the performance of batched inference is governed not just by how many requests are executed

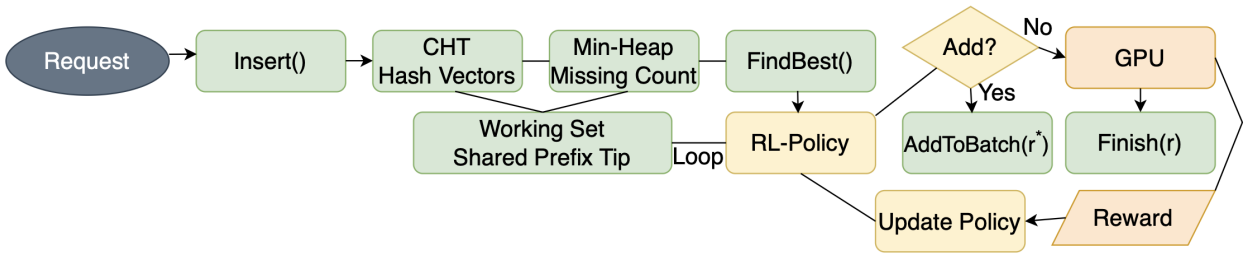


Figure 14. Entire Pipeline

together, but by a more subtle factor: how much prefix they share. When requests in a batch follow a common prefix, their KV-cache accesses become highly sequential, improving both temporal and spatial locality and enabling the hardware prefetcher to better utilize memory bandwidth. In contrast, even small deviations in prefix alignment can quickly fragment these access patterns, leading to degraded throughput.

This observation suggests that the length of the shared prefix across all active requests is a perfect metric for system performance. Longer shared prefixes directly translate into higher KV-cache reuse and more efficient memory access. However, this metric interacts in a non-trivial way with batch size: larger batches improve GPU utilization but often reduce prefix alignment.

4.1 Challenges and Key Ideas

Challenge 1: When to stop growing the batch? A natural strategy for improving throughput is to increase batch size. However, larger batches tend to include more heterogeneous requests, which reduces the shared prefix length and hurts memory locality. On the other hand, very small batches preserve strong prefix alignment but underutilize the GPU. Critically, existing schedulers do not address this tension: they greedily maximize batch size, filling batches up to a fixed capacity limit. Even schedulers that sort or group requests by prefix similarity [44, 46] still pack as many requests as possible, inevitably admitting heterogeneous requests that fragment prefix alignment once the pool of similar requests is exhausted.

As described in §3, moderately sized, highly homogeneous batches often outperform larger heterogeneous ones. The key question then becomes: what is the optimal batch size? More precisely, when should the scheduler stop adding requests to a batch? The answer depends on both the hardware (e.g., compute vs. memory balance) and the workload (e.g., prompt similarity), making any fixed heuristic inherently brittle.

Key Idea 1: Learn the stopping decision via RL. We model batch construction as a sequential decision process and use reinforcement learning (RL) to learn when to stop. At each step, the scheduler considers adding a new request and observes (batch size, shared prefix length, prefix after addition, future-compatible requests).

There are two signals that are particularly important: (i) **Shared prefix length:** the number of tokens common across all active requests. This directly captures temporal and spatial locality. (ii) **Future compatibility:** the number of waiting requests that share the same prefix up to the new level. This indicates how many future additions can be made at zero additional locality cost.

To build intuition, consider the following example. Suppose the current batch shares a prefix of length 10. A candidate request would reduce this to 8 (a loss of 2 tokens), which appears undesirable. However, if there are 5 additional waiting requests that also match this shorter prefix, then this one-time loss can be amortized over multiple future additions. In such cases, accepting a small immediate loss can lead to better overall throughput.

The RL policy learns to balance exactly these trade-offs, optimizing directly for decode throughput and adapting automatically to hardware and workload characteristics.

Challenge 2: Efficiently identifying prefix overlap. To make good scheduling decisions, the system must repeatedly answer: which waiting request shares the largest prefix with the current batch? Since scheduling decisions occur frequently and the waiting queue may contain thousands of requests with long prompts, detecting prefix overlap must be extremely fast and impose minimal CPU overhead.

Modern inference systems already exploit prefix sharing for KV-cache reuse. As shown in §2, vLLM [19] uses Merkle Trees, while SGLang [44] and BatchLLM [46] use radix trees to detect prefix sharing. Our scheduler, however, has a different objective: we do not need to store or manage KV-cache blocks; instead, we only need a lightweight way to estimate how much prefix overlap exists between requests so that we can select the best request to add to the batch. While a radix tree precisely captures prefix sharing, it incurs significant overhead; every insertion may trigger node splitting, and finding the best matching waiting request requires a DFS traversal that is $O(W)$ in the worst case, where W is the number of waiting requests. For a scheduler making frequent decisions over thousands of long-prompt requests, that is quite expensive. Similarly, Merkle-style hash chains

introduce unnecessary sequential dependencies during hash computation and expose more structure than the scheduler actually requires.

We start with a simple question: can we move away from trees? Instead of representing prefixes as paths in a tree, we divide each request into fixed-size chunks and compute prefix hashes at chunk boundaries. Each request is now represented as a sequence of hashes, and the active batch induces a working set of hashes currently in use. This reframes the problem from tree traversal to set overlap: which request has the maximum overlap with the current working set?

This shift eliminates structural overheads such as node splits and DFS traversal. An additional advantage of this design is that hash computation can be naturally batched and parallelized. Because chunks are processed independently, their hash contributions can be computed separately and later combined to obtain the cumulative prefix hashes. This pattern is similar in spirit to techniques such as online softmax, where partial statistics are computed over blocks and then merged to produce the final result. In contrast, Merkle-style hash chains require sequential computation because each hash depends on the previous one.

A straightforward solution is to scan all waiting requests and count how many of their hashes appear in the working set. The request with the highest overlap is selected. While correct, this approach requires $O(W \cdot C)$ work per decision (where C is the number of chunks), which is too expensive for high-frequency scheduling.

Key Idea 2: Incremental overlap tracking via missing counts. To avoid repeated rescanning, we maintain for each request a missing count: the number of its chunks not present in the working set.

Crucially, this count only changes when the working set changes (i.e., when requests are added to the batch or finished). Instead of recomputing overlap from scratch, we update these counts incrementally. This reduces the problem to finding the request with the smallest missing count.

Key Idea 3: Fast retrieval via lazy heap. To efficiently identify the best request, we maintain a min-heap over missing counts. When counts change, we push updated entries and use lazy deletion to skip stale ones.

This brings the selection cost down to $O(\log W)$ while keeping updates proportional only to affected requests.

The resulting design has two tightly coupled components: (i) A Chunked Hash representation that transforms prefix matching into a lightweight set-overlap problem and exposes both current locality (shared prefix length) and future opportunities (compatible requests). (ii) A learning-based scheduler that uses these signals to decide when to stop batch construction. Together, they enable the system to construct batches that are both compute-efficient and locality-aware.

4.2 System Overview

Figure 14 illustrates the overall pipeline of our scheduling system. Our is to construct batches that simultaneously (i) keep the GPU well utilized and (ii) maximize the shared prefix length across requests, thereby improving temporal and spatial locality.

System model. We consider an inference server that continuously receives requests. Each request consists of a prompt (input tokens) followed by autoregressive decoding. Execution proceeds in iterations, where each iteration processes either prompt tokens (prefill) or generates one token per active request (decode). At any time, requests are partitioned into three sets: (i) Waiting (W): requests that have not yet been scheduled, (ii) Active (A): requests currently being processed on the GPU, and (iii) Finished: requests whose generation has completed.

The scheduler operates in repeated execution steps. In each step, it incrementally constructs a batch by selecting requests from the waiting queue and activating them. Once the batch is finalized, it is executed on the GPU for one iteration, which may correspond to either processing prompt tokens (prefill) or generating the next token for active decoding requests.

Each request is represented as a sequence of C prefix hashes (chunks), derived from its token sequence of length N. The scheduler maintains a working set of hashes induced by the active set A, which defines the current shared prefix structure.

Unified request representation. All incoming requests are immediately converted into their chunked hash representation and inserted into the scheduler's data structures. Concretely, each request contributes: (i) its hash vector, (ii) entries in a reverse index mapping hashes to requests, and (iii) a missing count that tracks how much of its prefix is not yet covered by the current working set.

A global min-heap over missing counts allows the scheduler to efficiently retrieve the request with maximum overlap (i.e., minimum missing chunks) with the active batch.

Batch construction loop. The scheduler builds batches incrementally in a loop: (i) **Select candidate.** Using the heap, the scheduler retrieves the waiting request with the highest overlap with the current working set (via FindBest). (ii) **Compute metrics.** For this candidate, the system computes: current shared prefix length $|P|$, the prefix length after hypothetical addition, and the number of waiting requests that would remain compatible at that level. (iii) **Decision (RL policy).** These signals are passed to the policy, which decides whether to Add the request or Stop batch construction. (iv) **Update state.** If Add, the request is activated, the working set is updated, and overlap statistics are incrementally maintained. Otherwise, the batch is finalized.

This loop continues until the policy decides to stop or no suitable candidates remain.

Execution and iteration. Once a batch is finalized, it is dispatched to the GPU for one execution step. After execution, the completed requests are removed from the active set, updating the working set accordingly, and the batch construction resumes by repeating the same loop. This creates a continuous scheduling cycle where batches are formed, executed, and updated in an online fashion.

End-to-end flow. Putting everything together, the system operates as follows. Incoming requests are inserted into the chunked hash structure and indexed in the heap. The scheduler repeatedly selects the best candidate based on prefix overlap, queries the policy for a decision, and either activates the request or stops batch construction. The resulting batch is executed, and the observed throughput is used to refine future decisions. This process repeats continuously as new requests arrive and existing ones complete.

The following subsections describe the prefix-overlap data structure in detail, followed by the batching policies built on top of it.

4.3 Chunked Hash Tree

Before diving into the operations of the Chunked Hash Tree (CHT), we must define how requests are represented and how token sequences are converted into a compact representation that captures the prefix sharing structure. As shown in Figure 15, given a request with a token sequence $T = (t_1, t_2, \ldots, t_N)$, we partition T into non-overlapping chunks of size K (e.g., $K = 50$). For each chunk boundary $\ell = 1, \ldots C$ where $C = \lceil N/K \rceil$, we compute a cumulative prefix hash at each boundary

$h_\ell = \text{hash}(t_1, t_2, \ldots, t_{\min(\ell K, N)}), \quad \ell = 1, \ldots, C.$ [Do we need to highlight that when chunk size is 1, it is equivalent to radix tree, and otherwise approximate? We will do so in eval microbenchmark section too.]

Because the hash is cumulative, two requests share the same h_ℓ if and only if their first ℓK tokens are identical, which is precisely the condition under which their KV-cache prefixes can be shared. Each request r is therefore represented as an ordered hash vector $H_r = (h_1^r, h_2^r, \ldots, h_C^r)$. We use an incremental xxHash-64 state so that each token is processed exactly once, making the total cost $O(N)$ regardless of the number of chunked boundaries.

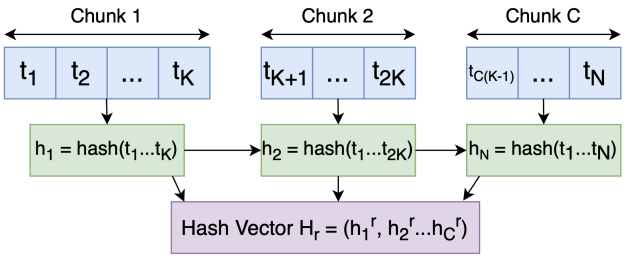


Figure 15. Hash Computation in Chunked Hash Tree

Inserting a request into Chunked Hash Tree. INSERT is invoked as soon as a new request r arrives at the scheduler. The goal is to register the request, compute its prefix hashes, and determine how well it aligns with the current working set.

We begin by computing the cumulative prefix-hash vector

$$H_r = (h_1^r, \ldots, h_C^r) \leftarrow \text{COMPUTEHASHES}(T, K),$$

and storing it in `request_hashes`, a map of request IDs to their hash vectors.

At the same time, we maintain a reverse index `waiting`, which maps each hash h to the set of waiting requests that contain h. As we iterate over H_r, we insert r into `waiting[`h`]` for every $h \in H_r$.

To evaluate how well r matches the current active batch, we use the `working_set`, which is defined as a set of tuples (ℓ, h) representing prefix hashes at specific levels that are currently covered by active requests.

Using this, we compute the missing count m_r, defined as the number of levels ℓ such that $(\ell, h_\ell^r) \notin \textit{working_set}$. Intuitively, m_r counts how many of r's prefix chunks are not already cached by the active batch. A request with $m_r = 0$ has its entire prefix covered by the current active batch and is therefore the most preferable candidate to admit next. We store this value in the `miss` map.

Finally, we insert the pair (m_r, r) into `waiting_heap`, a min-heap keyed by missing count. This allows efficient retrieval of the request that is most compatible with the current working set.

Importantly, INSERT does not modify reference counts or the working set. These are updated only when a request transitions from waiting to active.

Figure 16 shows an example of a chunked hash tree where we insert 3 requests, R_1, R_2, R_3. Requests R_1, R_2 share two levels, whereas R_3 just has level-1 in common with the other two. The heap has all 3 requests, each with a cost of 3.

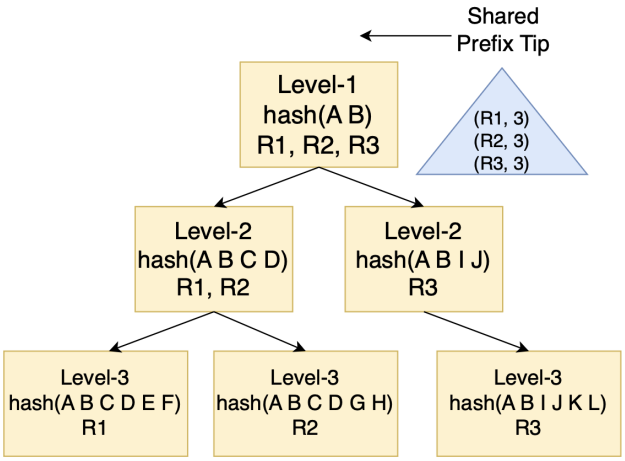


Figure 16. Insertion

Adding a request to the active batch. As proposed in §4.1, our key metric aims to find the number of chunks which are shared by all requests of the active batch. In the hash tree, this corresponds to a root-anchored path that extends downward as long as every active request agrees on the hash at each level, and stops at the first level of disagreement. Since this path always starts at the root and is contiguous, it is completely determined by where it ends. We therefore maintain a single tuple (ℓ^*, h^*) — the shared prefix tip — recording the depth and hash of its deepest node. An empty common prefix is denoted $\ell^* = 0$.

When request r is promoted into the active batch, two things must happen: the working set must be updated to reflect r's prefixes, and the shared prefix tip must be recomputed with r.

For the working set, any prefix hash of r that is newly entering the working set (ref count zero) is inserted, and every waiting request that shares that hash gets its miss count decremented — it now has one fewer prefix chunk to warm up. Reference counts are incremented for all levels regardless.

For the shared prefix tip, the key observation is that admitting a new request can only shorten the common prefix, never lengthen it. The new tip therefore lies at some level $\leq \min(\ell^*, |H_r|)$. To find it, we need the deepest level where r agrees with the existing tip hash and that node is in the working set — this is exactly the LCA of all active request leaves in the hash trie. Searching from the bottom up finds this immediately at the first hit, allowing an early stop.

Both updates are performed in a single backward pass over H_r. We iterate from level $|H_r|$ down to 1, updating the working set and miss counts at every level. Once we reach $\ell_{\text{tip}} = \min(\ell^*, |H_r|)$, we additionally begin checking for LCA agreement at each level. The first level where $h^r_\ell = h^*$ and $(\ell, h^r_\ell) \in \textit{working_set}$ is the new tip; we record it and continue the loop — without breaking — so that all remaining levels still receive their working set updates.

Figure 17 shows the chunked hash tree, after we activate request R_1. We mark the working set nodes as green. The shared prefix tip is set to level-2 hash of R_1, and the newer costs of R_2 and R_3 are appended to the waiting heap.

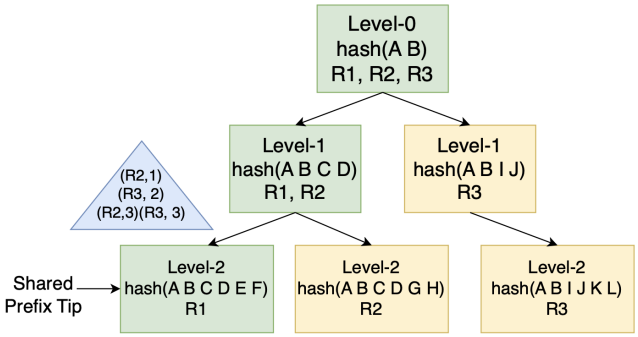


Figure 17. Adding a request to the active batch

Removing a request from the active batch. FINISH is called when request r completes. It is the symmetric counterpart of ADDTOBATCH: where activation could only shorten the shared prefix, completion can only lengthen it, since one fewer request needs to agree.

For the working set, we decrement the ref count of each hash in H_r. Any hash whose ref count drops to zero leaves the working set, and every waiting request containing that hash has its miss count incremented because that prefix chunk is no longer in the working set.

For the shared prefix tip, after removing r the remaining active requests may now agree on a longer common prefix than before. Starting from $\ell^* + 1$, we walk forward level by level, checking whether all remaining active requests share the same hash at each level. The first level of disagreement — or the shortest active request — ends the extension. This forward scan is cheap in practice since the tip rarely moves more than a few levels.

Both updates are performed in a single forward pass over H_r for the working set, followed by a short forward scan for the tip extension. The tip scan begins from $\ell^* + 1$ and consults the working set directly: a level ℓ can extend the tip only if (ℓ, h) is in the working set with ref count equal to $|\textit{active}|$, meaning every remaining active request carries that hash.

Finding the Best Request. The goal of FINDBEST is to identify the waiting request that would best complement the current active batch. The key insight is that the best candidate is the waiting request whose prefix chunks are most covered by the current active batch, i.e., the one with the smallest missing count m_r.

The primary selection criterion is the missing count m_r: the number of prefix chunks of r not yet in the working set. The waiting heap is a min-heap keyed by m_r, so the best candidate surfaces at the top. Since the heap uses lazy deletion, stale entries (where the recorded miss count no longer matches $miss[r]$) are skipped until a fresh entry is found.

Once the best candidate r^* is identified, we compute three statistics to inform the caller's admission decision. The first is ℓ^*_{before}, the current tip depth before admitting r^*. The second is ℓ^*_{after}, the tip depth that would result after admitting r^*, found by walking backward from $\min(\ell^*, |H_{r^*}|)$ until r^*'s hash matches the working set — the same LCA logic as AddToBatch, but without committing the update. The third is *peers*: the number of other waiting requests that already share the prefix down to depth ℓ^*_{after}, indicating how many future admissions would benefit at no additional tip cost.

The *peers* count captures an amortization argument. If admitting r^* shrinks the tip by $\Delta = \ell^*_{before} - \ell^*_{after} > 0$ levels, that is a one-time locality cost. But if $peers = k$ other waiting requests already share a prefix of depth ℓ^*_{after}, those k requests can be admitted immediately after at zero further tip cost — so the cost Δ is amortized over $k + 1$ admissions. FindBest surfaces this trade-off for free, leaving the admission policy a single focused question: is this cost worth it right now? How different policies answer that question is the subject of §4.4.

The result is cached and reused on repeated calls until the working set or waiting queue changes, avoiding redundant heap traversals.

An intuitive alternative to using the missing count is to simply select a waiting request that matches the current shared prefix tip (ℓ^*, h^*). The rationale is to maintain the current tip depth and prevent the shared chain from shrinking. However, this approach introduces a subtle flaw regarding future extension potential, because the tip is inherently bottlenecked by the most divergent active request. Consider a scenario where Level 0 is shared by all requests, but Level 1 splits into three distinct branches: one for R_1, one shared by R_2 and R_3, and a third for R_4. If our active batch contains R_1 and R_2, the shared prefix tip is restricted to Level 0. If we select the next request based solely on this current tip, R_3 and R_4 appear equally viable. Should the scheduler arbitrarily admit R_4, the tip remains at Level 0, but crucially, when R_1 finishes, the remaining requests (R_2 and R_4) sit on divergent branches, leaving the tip permanently stuck.

Our Chunked Hash Tree avoids this trap entirely by evaluating candidates against the complete `working_set` using the missing count m_r. Rather than anchoring to a bottlenecked tip, the working set tracks every prefix chunk kept alive by any active request. In the previous example, because R_2 is active, its Level 1 chunk is already present in the working set. Consequently, R_3 requires fewer missing chunks than R_4, resulting in $m_{R_3} < m_{R_4}$. By selecting R_3, the immediate tip remains bottlenecked at Level 0, but the scheduler implicitly preserves future overlap. Once R_1 finishes and is removed via Finish, the forward scan evaluates the remaining batch $\{R_2, R_3\}$ and successfully extends the shared prefix tip to Level 1, automatically reclaiming deeper locality benefits.

Figure 18 shows that R_2 was at the top of the waiting heap, and thus got selected as the best request. After adding R_2 to the active batch, it's nodes are appended to the working set, and the shared prefix tip is updated.

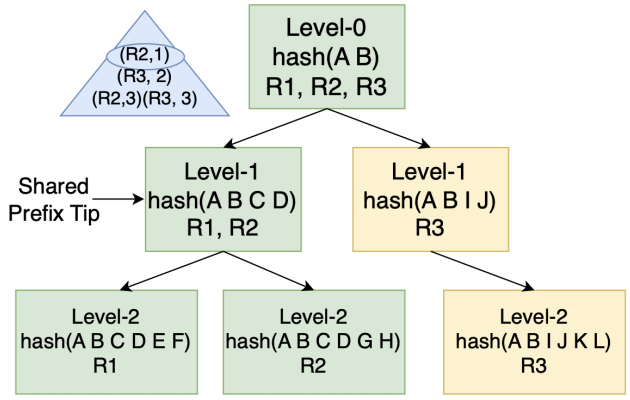


Figure 18. Finding the best request

Figure 19 shows the state of the chunked hash tree when R_1 finishes its execution. The shared prefix tip moves down to R_2's last level, and the costs in the waiting heap are updated accordingly.

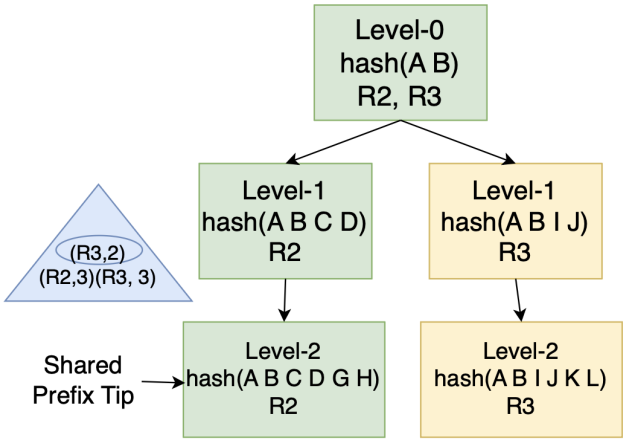


Figure 19. Request Completion

Optimizations.

Result caching. FindBest() may be called repeatedly within a single scheduling round without any intervening state change. We cache its last result — a four-tuple $(r^*, \ell^*_{before}, \ell^*_{after}, peers)$ — and return it immediately on subsequent calls until any mutation (an activation, completion, or new insertion) invalidates the cache flag.

Lazy heap updates. When a request's missing count changes, we push a fresh (m_r, r) entry onto the heap rather than updating the existing one in place. Stale entries — where the recorded m differs from $miss[r]$ — are silently skipped during FindBest. This avoids an $O(W)$ heap search on every miss-count change, keeping AddToBatch and Finish at $O(C \cdot W_h \cdot \log W)$, where W_h is the average number of waiting requests sharing a given hash.

Incremental shared prefix tip maintenance. Rather than recomputing the shared prefix tip from scratch after every state change — which would require scanning all active requests at $O(A \cdot C)$ cost — we exploit the asymmetry between the two events. Activation can only shorten the tip: AddToBatch finds the new tip via a single backward pass over H_r, terminating at the first working-set match. Completion can only lengthen the tip: Finish attempts a forward extension level by level beyond ℓ^*, using the ref count check to test agreement across all active requests, stopping at the first disagreement. Both updates are $O(C)$ in the worst case and terminate early in practice.

Pre-allocation. All major hash tables and vectors preallocate capacity at construction time, eliminating rehash latency and reallocation overhead during high-throughput operation.

Complexity Analysis. Table 2 summarizes the time complexity of each operation. T is the average number of tokens per request, C is the average number of chunks per request ($C = \lceil T/K \rceil$), W is the number of waiting requests, W_h is the average number of waiting requests sharing a given hash (typically $W_h \ll W$), and ℓ^* is the current tip depth.

Table 2. Time complexity of CHT scheduler operations.

Operation	Complexity
Insert	$O(T + C \log W)$
AddToBatch	$O(C \cdot W_h \cdot \log W)$
Finish	$O(C \cdot W_h \cdot \log W)$
FindBest	$O(\log W)$ amortized; $O(1)$ cached
Remove	$O(C)$

The tip update in both AddToBatch and Finish is subsumed within their $O(C \cdot W_h \cdot \log W)$ pass and adds no asymptotic overhead. The dominant cost in practice is the miss-count propagation in AddToBatch and Finish, which updates all waiting requests affected by each working-set change. Result caching eliminates redundant FindBest traversals between consecutive working-set mutations, keeping scheduling overhead low under sustained load.

4.4 Batching and Reinforcement Learning

any visualizations for these RL policies needed?

So far, the CHT provides us with an efficient way to find the best next request to add to the batch, the one that maximizes prefix reuse. But it does not tell us when to stop. Adding more requests always increases GPU utilization, but at some point, the shared prefix starts to shrink, hurting locality. This section describes how we frame this stop/continue decision as a learning problem and present three policies of increasing complexity: a hand-crafted heuristic that encodes domain intuition, a contextual bandit that learns from experience without temporal structure, and a Q-learning agent that reasons about long-term throughput.

As highlighted in §4.2, the scheduler incrementally builds a decoding batch by repeatedly selecting the lowest-cost waiting request via the CHT and deciding whether to add it to the current batch or dispatch the batch as-is. This stop/continue decision is the crux of the batching problem.

State, Actions, and Reward. At each step of batch formation, the scheduler observes a state

$$s = (batch_size,\ chunks_before,\ chunks_after,\ requests_waiting)$$

and takes a binary action $a \in \{\text{Add}, \text{Stop}\}$. Here, *batch_size* denotes the number of requests that are already in the batch, which is a proxy for current GPU utilization. *chunks_before* is the shared-prefix chain length before adding the candidate request; *chunks_after* is the chain length after hypothetically adding it (obtained from FindBest); and *requests_waiting* is the number of waiting requests that share the prefix up to level *chunks_after*, indicating whether the locality cost of this addition will be compensated by future activations. Together, these four values capture both the current GPU utilization (batch size) and the cost of the proposed addition to prefix locality, summarized by the chunk loss $\Delta = chunks_before - chunks_after$).

Both learning policies share the same reward signal: the decode throughput of the dispatched batch,

$$\mathcal{R} = \text{throughput}_{\text{decode}}$$

Maximizing this reward simultaneously encourages large batches (better GPU utilization) and short decode times (better prefix locality). Both policies also operate over the same discretized state space: *batch_size* and *requests_waiting* are mapped to exponential bins, while the chunk loss Δ is mapped to a coarser set of thresholds. The discretized state is a triple $(\hat{b}, \hat{\Delta}, \hat{w})$, where each hat denotes the bin index. The resulting table is small enough to store in a hash map and converges quickly, even under moderate traffic.

Heuristic Policy. Before introducing any learning, we establish a baseline that encodes domain intuition directly as rules. The heuristic is designed around two competing goals: maximizing GPU utilization by filling the batch as much as possible and preserving prefix locality by avoiding candidates that cause large drops in the shared prefix chain. The heuristic always adds a candidate that does not hurt the shared chain at all—a free addition. When the batch is still very small and GPU utilization is clearly low, it tolerates a slightly larger locality cost to avoid dispatching an underutilized batch. In the general case, it accepts a candidate only if the chunk loss is within a small tolerance. Finally, when many waiting requests share the same prefix level, it allows a marginally higher locality cost since the short-term penalty is likely to be offset by the homogeneous additions that follow. If none of these conditions are met, the heuristic stops and dispatches the current batch.

This policy requires no training and converges immediately, but its thresholds are fixed and cannot adapt to shifts in workload distribution.

Contextual Bandit. To move beyond hand-tuned thresholds, we implement a contextual bandit using Upper Confidence Bound (UCB) selection [6]. The bandit treats each batch-construction call as an independent decision: given the discretized state $\hat{s} = (\hat{b}, \hat{\Delta}, \hat{w})$, it selects the action that maximizes

$$\mathrm{UCB}(\hat{s}, a) = \frac{\mu(\hat{s}, a)}{n(\hat{s}, a)} + c\sqrt{\frac{\ln N}{n(\hat{s}, a)}}$$

where $\mu(\hat{s}, a)$ is the cumulative reward for taking action a in state $\hat{s}$, $n(\hat{s}, a)$ is the number of times that pair has been selected, N is the total number of steps, and c is the exploration coefficient. The first term favors actions that have historically yielded high throughput; the second term ensures that under-explored actions are tried sufficiently often.

Because the bandit discards temporal structure entirely, it does not reason about how today's batching decision affects the state of the queue for the next batch, it can miss opportunities where a short-term locality sacrifice leads to a better-amortized sequence of future batches. This motivates the full reinforcement learning approach.

Q-Learning Policy. To capture the temporal dependencies that the bandit ignores, we model batch construction as an episodic reinforcement learning problem. A new episode begins each time the scheduler starts forming a fresh batch and terminates when it issues Stop or the waiting queue is exhausted. The Q-table [34] stores a value $Q(s, a)$ for every (discretized state, action) pair and is updated upon each batch completion via the standard Bellman update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$$

where r is the observed throughput reward, s' is the state at the start of the next batch formation, α is the learning rate, and γ is the discount factor. The discount factor encodes that the agent values near-term throughput slightly more than long-term gains, which matches the real-time nature of an inference server. During training, ε starts at 1.0 (fully random) and decays linearly to $\varepsilon_{\min}$ over a configurable number of steps, after which the policy is nearly deterministic.

The BuildBatch Procedure. Algorithm 1 shows how all three policies fit into the same batch-construction loop. At each iteration, FindBest (Algorithm 6) is called to retrieve the lowest-cost candidate from the CHT along with its chain statistics c_{before}, c_{after}, and the number of waiting requests w that share the same prefix level. These four values are assembled into the state s and passed to whichever policy π is active—heuristic, bandit, or Q-learning agent—which returns either Add or Stop. If Add, the candidate is activated and appended to the batch; if Stop, the loop exits and the batch is dispatched immediately. Once the batch executes, the observed decode throughput serves as the reward signal: the bandit updates its count-and-mean tables, while the Q-learning agent applies the Bellman update above. The heuristic requires no update.

Algorithm 1 BuildBatch()

Require: Waiting queue Q; policy π (heuristic, bandit, or Q-learning); discount γ; learning rate α
Ensure: Batch B ready for one decode iteration

$B \leftarrow \emptyset$
while $Q \neq \emptyset$ **do**
  $(r^*, c_{\text{before}}, c_{\text{after}}, w) \leftarrow$ FindBest()
  if $r^* = \perp$ **then break**
  end if
  $s \leftarrow (|B|,\ c_{\text{before}},\ c_{\text{after}},\ w)$
  $a \leftarrow \pi(s)$ ▷ heuristic rule / UCB look-up / ε-greedy Q-table
  if $a =$ Stop **then break**
  end if
  Activate(r^*); $B \leftarrow B \cup \{r^*\}$
end while
$\text{throughput}_{\text{decode}} \leftarrow$ ExecuteBatch(B)
Update π with observed reward $\text{throughput}_{\text{decode}}$ ▷ no-op for heuristic
return B

4.5 Integration with vLLM and SGLang

We integrate our scheduler as a lightweight layer on top of both vLLM's [19] and SGLang's [44] existing request management pipeline. The Chunked Hash Tree (implemented in C++), shared prefix chain tracking, and reinforcement learning policies operate entirely at the scheduler level.

5 Evaluation

We evaluate Feather across a range of models, GPU configurations, and real-world datasets. Our evaluation seeks to answer the following questions:

1. How does Feather's performance scale across diverse workloads, varying numbers of prefix groups, decode lengths, and input sequence lengths?
2. What gains does Feather deliver for both online and offline serving under real-world workloads?
3. What is the contribution of each component of Feather to overall performance?
4. How much does replacing individual components with ideas from prior work degrade throughput?
5. What is the computational overhead introduced by Feather's scheduling?

5.1 Experimental Setup

Models and Hardware. We evaluate on three model families spanning a range of sizes: Qwen [7] (0.5B,1.5B,8B) and LLaMA-3 [3] (8B). Unless stated otherwise, all experiments run on an NVIDIA RTX 6000 Ada Generation GPU [24] with 48 GB of GDDR6 and a 96 MB L2 cache, with the maximum batch size fixed at 500. [Present results on A100 for PAT? Any other GPU?]

Workloads. Following prior work, request arrivals are modeled as a Poisson process, with input and output lengths sampled from two real-world datasets. L-Eval [4] is a benchmark of human-labeled query-response pairs covering diverse tasks such as summarization and question answering, with sequence lengths ranging from 2.7K to 210.5K tokens. LongBench [8] is a curated collection of long-context samples drawn from 21 datasets across 6 task categories—including multi-document question answering, summarization, and code completion—with context lengths between 4K and 10K tokens.

Baselines. We compare Feather against the following state-of-the-art LLM serving systems, all with prefix caching enabled by default. vLLM [19] is evaluated under its default FCFS scheduling policy. SGLang [44] is evaluated under all three of its supported scheduling policies—FCFS, LPM, and DFS–W—to provide a comprehensive comparison across its design space. Finally, DynamicBatching [29] represents a recent approach specifically optimized for LLM inference batching.

Metrics. We report three key metrics throughout our evaluation. Throughput (toks/s) measures the number of output tokens generated per second, reflecting overall serving capacity and GPU utilization. Time Between Tokens (TBT) captures the average time elapsed between consecutive decoded tokens, reflecting the smoothness and responsiveness of generation—a metric of particular importance in interactive serving settings. Finally, we also track the average batch size per forward pass, which helps to understand the relationship between scheduling decisions and the above two metrics.

5.2 End-to-End Comparison

Throughout this section, we evaluate a workload consisting of 5 prefix groups, each of length 5000 tokens, with roughly equal request distribution across groups. Unless otherwise stated, the input request rate is 500 req/sec, each request generates 200 output tokens, and experiments run on the Llama3-8B model [3]. We vary each of these configuration parameters in turn to examine how Feather compares against all baselines. The maximum batch size is fixed at 500, and the token budget is set at 32,768 tokens.

Varying Input Request Rate. Figure 20 presents throughput (toks/s) as a function of the input Poisson rate. Across all policies, throughput rises with increasing request rate before eventually saturating. Feather consistently outperforms all baselines at every request rate. At 100 req/sec, Feather achieves roughly 4× the throughput of vLLM's FCFS policy.

Notably, this throughput advantage is not driven by larger batch sizes. As shown in Figure 21, Feather maintains a smaller average batch size than vLLM:FCFS across all request rates. This reinforces a key insight: smaller, homogeneous batches outperform larger, heterogeneous ones. The gradual growth in Feather's batch size with increasing request rate reflects its ability to smartly (better adverb here) aggregate more requests from the same prefix group into the active batch without sacrificing locality. We also observe that SGLang policies consistently operate at lower batch sizes than vLLM policies; this is because vLLM avoids double-counting shared prefix tokens toward the token budget, allowing it to pack more requests into each batch. Dynamic Batching (denoted as DB on the plot) performs quite similar to vLLM FCFS, because all the KV caches of the exeperimental setup fit onto the GPU memory.

Turning to time-between-tokens (TBT) in Figure 22, Feather achieves substantially lower TBT than all baselines except SGLang:FCFS, whose smaller batch sizes naturally yield faster per-token generation. While SGLang's LPM and DFS-W policies also operate at low batch sizes, their TBT remains significantly higher than Feather's - a consequence of the non-trivial scheduling overhead incurred during their batch construction process (see §for a detailed breakdown). Together, these results demonstrate that Feather strikes an effective balance between batch efficiency, cache locality, and scheduling overhead.

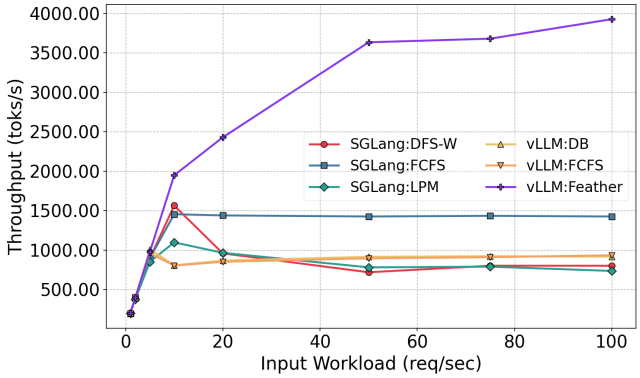


Figure 20. Throughput across varying input workload Poisson rates.

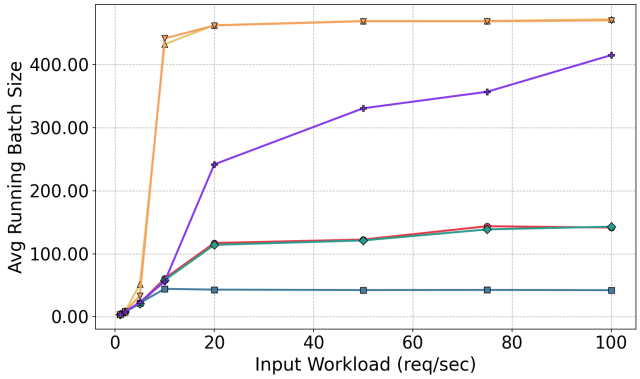


Figure 21. Average batch size across varying input workload Poisson rates.

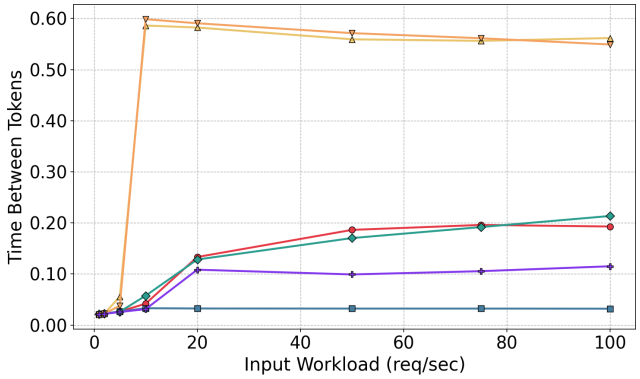


Figure 22. Time-between-tokens (TBT) across varying input workload Poisson rates.

Varying Number of Prefix Groups. Figures 23, 24, and 25 present throughput, average batch size, and TBT as a function of the number of prefix groups, respectively.

As established in §3, increasing the number of prefix groups introduces request heterogeneity, which degrades both temporal and spatial locality in the KV cache. This manifests as a sharp drop in throughput under vLLM's FCFS policy even when moving from just 1 to 2 prefix groups. As the number of groups grows further, FCFS continues to suffer: beyond 20 prefix groups, the aggregate KV cache footprint exceeds HBM capacity, causing a pronounced collapse in average batch size (Figure 24) and a corresponding increase in recomputation, which further depresses throughput.

Feather mitigates this by co-scheduling requests that share a prefix group, improving cache reuse and reducing evictions. This allows Feather to sustain a larger effective batch size and significantly outperform all baselines in throughput across the entire range of prefix group configurations.

SGLang's LPM and DFS-W policies occupy a middle ground: their prefix-aware scheduling reduces KV cache evictions and supports larger batch sizes relative to FCFS, yielding higher throughput than vLLM:FCFS at scale. However, the additional per-step scheduler overhead inflates their TBT, making them less suitable when latency is a primary concern. Overall, Feather achieves the best throughput across all settings while keeping TBT competitive.

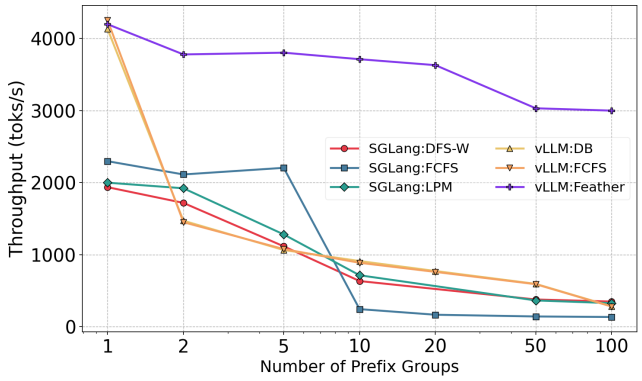


Figure 23. Throughput across varying number of prefix groups.

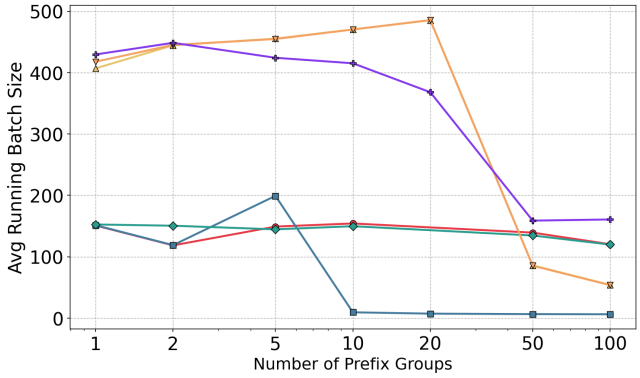


Figure 24. Average running batch size across varying number of prefix groups.

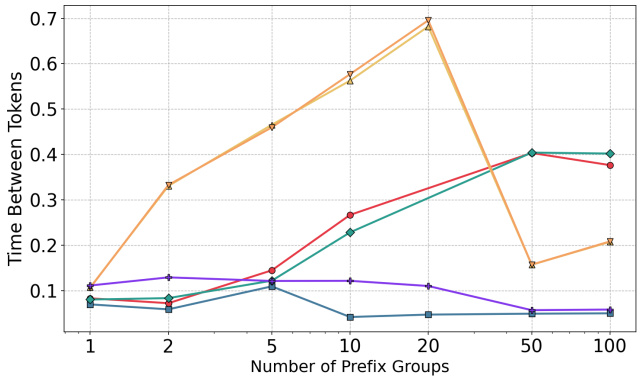


Figure 25. Time-between-tokens (TBT) across varying number of prefix groups.

Varying Number of Decode Tokens. Figures 26, 27, and 28 present throughput, average batch size, and TBT as a function of the number of output decode tokens per request. Each decode token generation corresponds to a sweep through its entire KV cache. The policies exhibit two qualitatively distinct regimes separated by a crossover around 20 decode tokens. In the prefill-dominated regime (1–20 tokens), all policies behave similarly: throughput grows modestly and batch sizes converge, since the workload is dominated by short prefill phases that leave little room for locality-aware scheduling to take effect.

Beyond 20 tokens, the policies diverge sharply. FEATHER scales strongly with decode length, reaching roughly 3800 toks/s at 200 tokens — more than 3× the throughput of any SGLang policy at the same point. Two complementary effects drive this. First, longer decode phases mean each batch sweeps through the KV cache repeatedly, so the gains from temporal and spatial locality compound over more iterations, allowing FEATHER to utilize the benefits of locality for a larger fraction of total execution time. Second, longer decodes give the scheduler more opportunities to accumulate waiting requests from the same prefix group, enabling it to assemble larger, more homogeneous batches. Both effects reinforce each other as decode length grows, explaining the near-monotonic rise in FEATHER's average batch size and throughput.

SGLang policies follow the opposite trajectory in batch size. Because SGLang counts shared prefix tokens toward the token budget, each prefill contributes roughly 5000 tokens to the overall token budget, while each decode step contributes only a single token. As requests become decode-heavy, fewer prefills occur per scheduling step, meaning fewer new requests are available at each step to populate the batch. This causes the effective batch size to collapse for SGLang policies (Figure 27). The resulting smaller batches reduce per-step compute, which drives down TBT for SGLang:FCFS and LPM, but at the cost of throughput, since fewer tokens are generated per second. Although FEATHER operates at a higher batch size than SGLang policies, its TBT remains competitive: reduced scheduler compute overhead and the gains from temporal and spatial locality together offset the cost of processing larger batches.

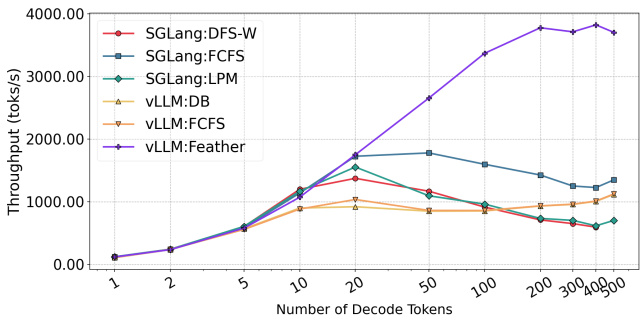


Figure 26. Throughput across varying number of decode tokens.

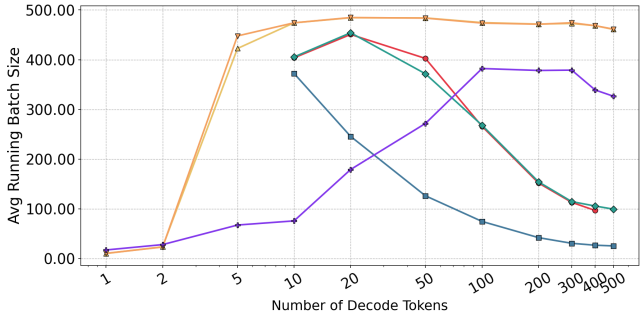


Figure 27. Average running batch size across varying number of decode tokens.

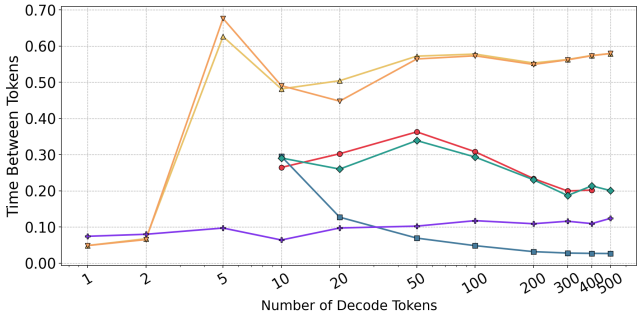


Figure 28. Time-between-tokens (TBT) across varying number of decode tokens.

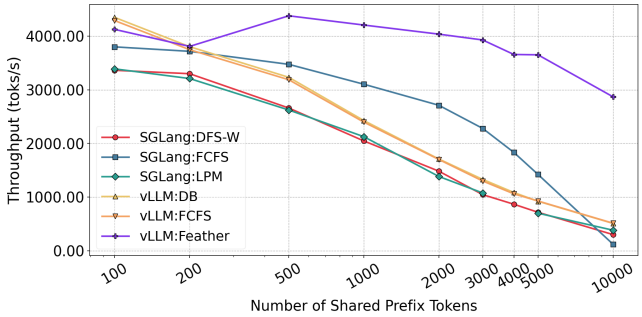


Figure 29. Throughput across varying number of shared prefix tokens.

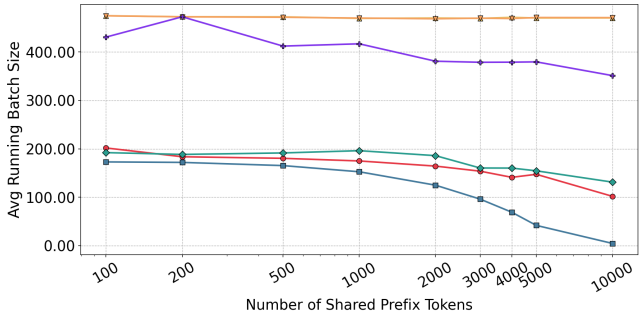


Figure 30. Average running batch size across varying number of shared prefix tokens.

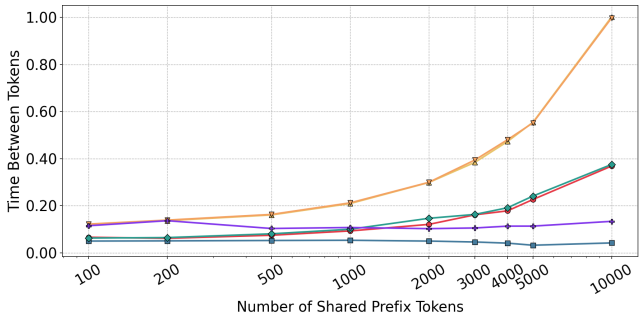


Figure 31. Time-between-tokens (TBT) across varying number of shared prefix tokens.

Varying Number of Shared Prefix Tokens. Figures 29, 30, and 31 present throughput, average batch size, and TBT as the shared prefix length increases from 100 to 10000 tokens. The overall trend is expected: longer prefixes increase prefill compute and KV cache memory pressure, dragging down throughput across all policies. What is more revealing is how differently the policies respond to this pressure.

At short prefix lengths, all policies perform similarly and throughput is comparable across the board. Beyond 1000 tokens, the policies diverge sharply. FEATHER's throughput advantage grows with prefix length, a direct consequence of higher locality gains as sequences get longer. vLLM:FCFS, despite maintaining large batch sizes throughout (owing to its idea of excluding shared prefix tokens from the token budget), sees its throughput collapse and TBT increase exponentially to 1.0s at 10000 tokens because packing oversized, heterogeneous batches destroys locality. FEATHER avoids this by running moderately sized but homogeneous batches, keeping TBT nearly flat across the entire range. The SGLang policies tell a different story: because SGLang counts prefix tokens toward the token budget, longer prefixes spread out requests and batch sizes reduce by 10000 tokens. SGLang:FCFS preserves low TBT as a side effect of these tiny batches, but at a steep throughput cost. SGLang:LPM and DFS-W suffer on both fronts, with TBT rising steadily due to heavier memory access and growing CPU scheduling overhead for longer prompts.

Taken together, as shared prefix lengths grow, FEATHER is the only policy that sustains both high throughput and low latency, while all baselines are forced to sacrifice one for the other.

Varying Models. Figures 32, 33, and 34 present throughput, average running batch size, and time-between-tokens (TBT) across models of varying sizes. The overall trend is consistent with expectations: smaller models achieve higher throughput owing to lower per-token compute cost. FEATHER outperforms all baselines across all models. The sole exception is Qwen-1.5B, where SGLang FCFS achieves marginally higher throughput; however, FEATHER compensates on the latency axis, exhibiting substantially lower TBT for the same model.

A key structural observation is that Feather's relative throughput advantage over the baselines grows with model size. As model size increases, a larger fraction of total execution time is spent in attention computation, which amplifies the benefits of temporal and spatial KV-cache locality that Feather exploits. This effect is visible in Figure 32: the gap between Feather and the next-best baseline widens from Qwen-0.5B to the 8B-scale models.

The performance of SGLang's LPM and DFS-W schedulers degrades noticeably for smaller models. Both policies rely on CPU-side tree traversal whose overhead scales with sequence length rather than model size; as the model shrinks, GPU execution time decreases while scheduler overhead remains roughly constant, causing the CPU to become an increasing bottleneck. Feather avoids this through its chunked hash tree, which reduces per-step scheduler complexity and keeps CPU overhead negligible relative to GPU execution time across all model scales.

Finally, Figure 34 highlights that vLLM's FCFS and DB baselines incur substantially higher TBT at larger model sizes, whereas Feather keeps TBT low and stable.

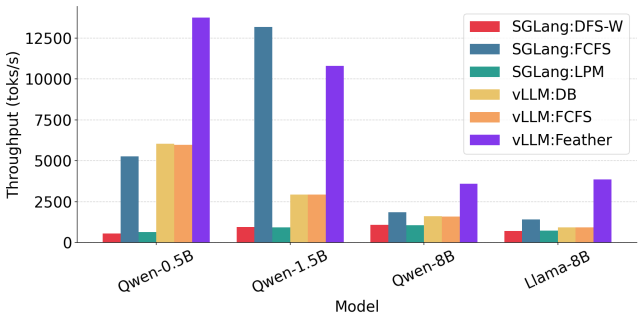


Figure 32. Throughput (tokens/s) across models of varying size.

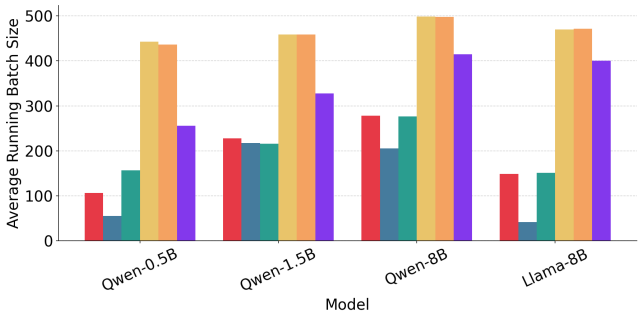


Figure 33. Average running batch size across models of varying size.

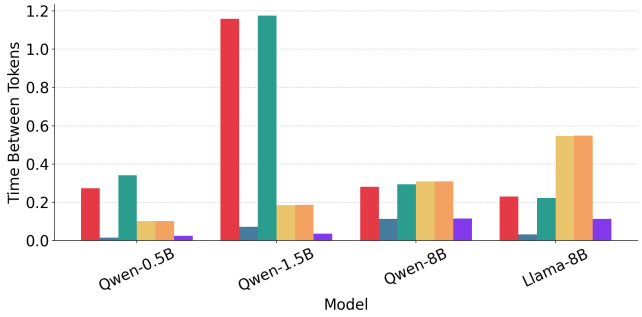


Figure 34. Time-between-tokens (TBT) across models of varying size.

5.3 Micro-Benchmarks

Comparing Scheduler Compute Overhead across Different Policies. Figure 35 shows the CPU scheduling overhead for each policy as the shared prefix length varies from 100 to 20,000 tokens, plotted alongside actual GPU execution time to give a meaningful frame of reference.

We include an Oracle policy as a baseline, where the caller explicitly supplies prefix group identifiers with each request. This lets the scheduler batch requests sharing the same prefix instantly, with no matching computation, effectively zero overhead while still realizing all the cache-locality benefits that motivated Feather. A second Oracle line shows the GPU execution time under this ideal batching, representing the throughput ceiling any policy could hope to achieve.

All three non-trivial policies, Feather, LPM, and DFS-W show increasing overhead as prefix length grows, since longer contexts mean deeper radix-tree traversals and larger key comparisons. But the rate of growth diverges dramatically. At 20,000 tokens, LPM and DFS-W overhead is roughly 1,000× higher than Feather's, and at larger sequence lengths their scheduling overhead exceeds total Oracle GPU execution time by up to 10×, meaning the CPU bottleneck alone is enough to negate any gains from cache-aware batching. Feather, by contrast, stays below 1% of GPU execution time across the entire tested range.

Let N be the number of requests in the waiting queue and L the maximum prefix length. For each request, LPM performs prefix matching on the global radix cache tree, a token-by-token traversal costing $O(L)$ per request, then sorts the queue by matched prefix length, adding $O(N \log N)$. Total per-round cost is $O(N \cdot L + N \log N)$. The in-batch prefix caching check adds another $O(N \cdot L)$ term. The result is overhead that scales multiplicatively with both queue size and context length. LPM's own implementation implicitly acknowledges this by falling back to FCFS when $N > 128$ (we disabled the fallback during these experiments).

DFS-W avoids per-request tree traversal, but replaces it with something different: a full traversal of the entire radix cache tree every scheduling round. Let T be the total node

count. DFS-W first propagates subtree request counts upward in $O(T)$, then re-traverses depth-first, sorting children by weight at every level. Total cost is $O(N+T \log T)$. The key problem is that T reflects the server's accumulated history, not the current workload. The radix tree grows monotonically and is only pruned under memory pressure, so DFS-W's overhead increases indefinitely over a deployment's lifetime, regardless of current queue length or request size.

These failure modes compound adversarially in practice. A server with a large accumulated cache ($T \gg N$) will punish DFS-W even during quiet periods; a burst of long-context requests will punish LPM through the $N \cdot L$ term. Either way, scheduling overhead can reach the same order of magnitude as GPU execution time. The broader takeaway is that algorithmically identifying shared prefixes at runtime is fundamentally expensive and that naive cache-aware scheduling can itself become the dominant bottleneck.

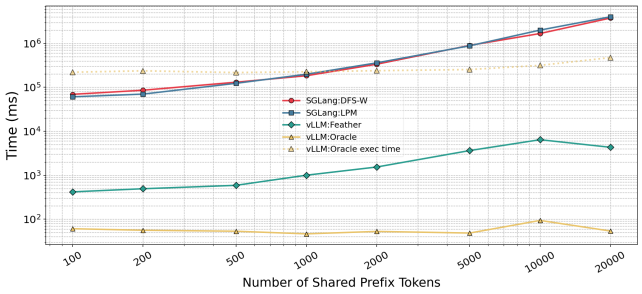


Figure 35. CPU scheduling overhead (in milliseconds) for Feather, LPM, DFS-W, and Oracle policies as a function of shared prefix length (100 to 20,000 tokens). The dotted line denotes the total GPU execution time under the Oracle policy.

Time taken by individual functions of Chunked Hash Tree. Table 3 profiles the CHT operations over an inference workload of 5,000 requests, each 5,000 tokens long. As expected, Insert, AddToBatch, and Finish are each invoked exactly 5,000 times, once per request lifecycle. FindBest, however, is called 26,405 times, since it is queried repeatedly during batch construction: the RL policy may reject a candidate and trigger a fresh lookup, or the scheduler may inspect the queue multiple times before committing to a Stop.

Insert dominates total time, consuming ~768 ms out of the ~1,194 ms aggregate. This is expected: hash computation must scan every token once ($O(N)$ per request), making Insert inherently proportional to prompt length. Finish is notably more expensive than AddToBatch (~195 ms vs. ~56 ms) despite their symmetric nature. The gap arises from Finish's additional forward tip-extension scan: after updating the working set, it must walk forward from ℓ^*+1 level by level to check whether the shared prefix can be lengthened, an overhead that AddToBatch's early-terminating backward LCA scan does not incur. FindBest is the cheapest per-call operation at 6,641 ns on average, a direct consequence of result caching and lazy heap traversal — its high call count therefore contributes only a smaller chunk to the total overhead (~175 ms).

Function	Number of Calls	Average Time (ns)	Total Time (ns)
Insert	5,000	153,714	768,570,573
AddToBatch	5,000	11,143	55,715,469
Finish	5,000	38,952	194,760,111
FindBest	26,405	6,641	175,372,986

Table 3. Time taken by individual functions of Chunked Hash Tree for 5,000 requests of 5,000 tokens each.

Mean DRAM Activity across various sequence lengths. Figure 36 presents the mean DRAM activity for various policies across different shared prefix lengths. As discussed in §3, homogeneous batches promote higher temporal and spatial locality, which in turn increases prefetcher activity and leads to more effective utilization of available DRAM bandwidth. This is clearly reflected in Feather's consistently higher mean DRAM activity compared to all other policies across nearly the entire range of prefix lengths tested.

There is a pronounced and consistent decreasing trend in mean DRAM activity for SGLang:DFS-W, SGLang:LPM, and vLLM:DB as the number of shared prefix tokens increases. SGLang:LPM and SGLang:DFS-W exhibit the steepest decline, dropping from ~0.156 and ~0.146 at 100 tokens to just ~0.037 and ~0.043 respectively at 10,000 tokens—a reduction of more than 3×. This degradation stems from two compounding factors: first, these policies do not halt batch formation at the right moment, resulting in heterogeneous batches that disrupt spatial and temporal locality; second, they incur large CPU stalls that further reduce effective memory utilization. vLLM:DB, while more stable than the SGLang variants at longer prefix lengths, still lags significantly behind Feather.

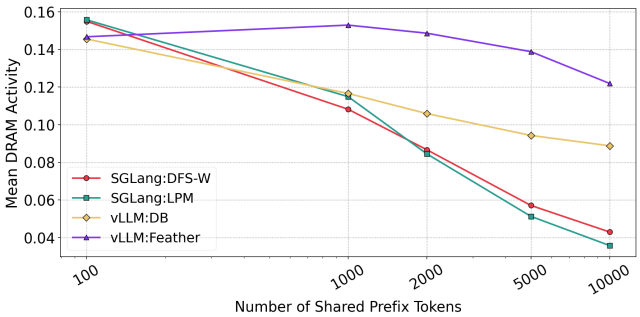


Figure 36. Mean DRAM Activity for various policies as a function of shared prefix length (100 to 10,000 tokens).

How to compare the two RL policies?

Effect of Chunk Size.

5.4 Ablation Study

What if we replace the Chunked Hash Tree with a Radix Tree? Figures 37 and 38 compare the throughput and time-between-tokens (TBT) when the Chunked Hash Tree (CHT) in Feather is substituted with a token-level radix tree, while keeping the reinforcement learning (RL)-based batch formation policy unchanged.

Across all shared prefix lengths, the radix tree variant achieves substantially lower throughput than Feather. Although the RL policy continues to ensure that batches consist of homogeneous requests, the significantly higher CPU overhead introduced by the radix tree causes pronounced stalls between consecutive GPU iterations. This is directly reflected in the TBT results: the radix tree consistently exhibits much higher inter-token latency, indicating that the GPU frequently sits idle while the CPU scheduler catches up. The CHT's chunk-level granularity reduces this overhead substantially, enabling Feather to sustain high GPU utilization and deliver better end-to-end throughput.

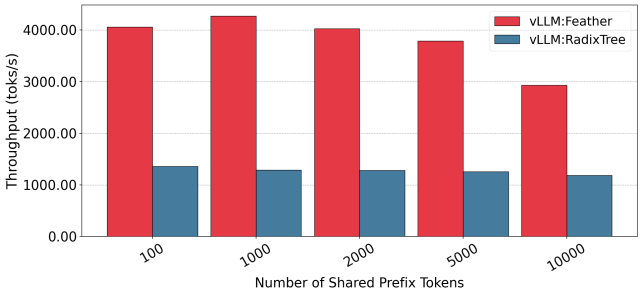


Figure 37. Throughput vs. number of shared prefix tokens: Chunked Hash Tree (Feather) vs. token-level radix tree.

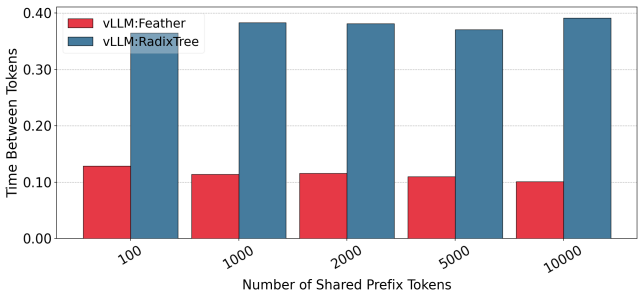


Figure 38. Time-between-tokens (TBT) vs. number of shared prefix tokens: Chunked Hash Tree (Feather) vs. token-level radix tree.

What if we remove RL?.

6 Conclusion

References

[1] 0xeb. 2024. The Big Prompt Library: A Collection of Prompts, System Prompts and LLM Instructions. https://github.com/0xeb/TheBigPromptLibrary. GitHub repository.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. arXiv:2403.02310 [cs.LG] https://arxiv.org/abs/2403.02310

[3] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[4] Chenxin An, Shansan Gong, Ming Zhong, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2023. L-Eval: Instituting Standardized Evaluation for Long Context Language Models. arXiv:2307.11088 [cs.CL]

[5] Anthropic. 2024. Claude. https://claude.ai.

[6] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 2-3 (2002), 235–256. doi:10.1023/A:1013689704352

[7] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv preprint arXiv:2309.16609 (2023).

[8] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. arXiv preprint arXiv:2412.15204 (2024).

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[10] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. arXiv:2307.13304 [cs.LG] https://arxiv.org/abs/2307.13304

[11] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. 2024. A Complete Survey on LLM-based AI Chatbots. arXiv:2406.16937 [cs.CL] https://arxiv.org/abs/2406.16937

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

[13] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin

Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[14] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. 2023. Towards Next-Generation Intelligent Assistants Leveraging LLM Techniques. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23). Association for Computing Machinery, New York, NY, USA, 5792–5793. doi:10.1145/3580305.3599572

[15] Google. 2024. Gemini. https://gemini.google.com.

[16] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[17] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-A <scp>ttention</scp> : Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25). ACM, 897–912. doi:10.1145/3676641.3715996

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] https://arxiv.org/abs/2309.06180

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL] https://arxiv.org/abs/2005.11401

[22] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978 [cs.CL] https://arxiv.org/abs/2306.00978

[23] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2025. Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing. arXiv:2504.19516 [cs.DC] https://arxiv.org/abs/2504.19516

[24] NVIDIA Corporation. 2024. NVIDIA RTX 6000 Ada Generation Datasheet. https://resources.nvidia.com/en-us-briefcase-for-datasheets/proviz-print-rtx6000-1?ncid=no-ncid Accessed: January 25, 2026.

[25] OpenAI. 2024. ChatGPT. https://chatgpt.com.

[26] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng,

Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[27] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In Proceedings of Machine Learning and Systems, M. Zaharia, G. Joshi, and Y. Lin (Eds.), Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/96894468eb44631a32d7ebd56f9892c7-Paper-Conference.pdf

[28] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. arXiv:2503.05248 [cs.DC] https://arxiv.org/abs/2503.05248

[29] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. arXiv preprint arXiv:2503.05248 (2025).

[30] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. 2021. True Few-Shot Learning with Language Models. In Advances in Neural Information Processing Systems, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 11054–11070. https://proceedings.neurips.cc/paper_files/paper/2021/file/5c04925674920eb58467fb52ce4ef728-Paper.pdf

[31] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. arXiv:2405.04437 [cs.LG] https://arxiv.org/abs/2405.04437

[32] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-throughput LLM Inference via Model Re-sharding. arXiv:2503.06433 [cs.DC] https://arxiv.org/abs/2503.06433

[33] vLLM Project Contributors. 2024. Automatic Prefix Caching. https://docs.vllm.ai/en/stable/design/prefix_caching/. vLLM Documentation. Accessed: 2026-03-10.

[34] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-Learning. Machine Learning 8, 3-4 (1992), 279–292. doi:10.1007/BF00992698

[35] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. arXiv:2206.07682 [cs.CL] https://arxiv.org/abs/2206.07682

[36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. Transactions on Machine Learning Research (2022). https://openreview.net/forum?id=yzkSU5zdwD Survey Certification.

[37] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. arXiv:2404.09526 [cs.DC] https://arxiv.org/abs/2404.09526

[38] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. 2025. DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference. arXiv:2404.00242 [cs.CL] https://arxiv.org/abs/2404.00242

[39] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444 [cs.LG] https://arxiv.org/abs/2405.16444

[40] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. arXiv:2402.15220 [cs.LG] https://arxiv.org/abs/2402.15220

[41] Jinjun Yi, Zhixin Zhao, Yitao Hu, Ke Yan, Weiwei Sun, Hao Wang, Laiping Zhao, Yuhao Zhang, Wenxin Li, and Keqiu Li. 2025. PAT: Accelerating LLM Decoding via Prefix-Aware Attention with Resource Efficient Multi-Tile Kernel. arXiv:2511.22333 [cs.DC] https://arxiv.org/abs/2511.22333

[42] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. arXiv:2503.18292 [cs.DC] https://arxiv.org/abs/2503.18292

[43] Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yang Zhou, Jiarong Xing, and Ion Stoica. 2024. BlendServe: Optimizing Offline Inference for Auto-regressive Large Models with Resource-aware Batching. arXiv:2411.16102 [cs.LG] https://arxiv.org/abs/2411.16102

[44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[45] Wanyi Zheng, Minxian Xu, Shengye Song, and Kejiang Ye. 2025. BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving. arXiv:2507.17120 [cs.DC] https://arxiv.org/abs/2507.17120

[46] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2025. BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching. arXiv:2412.03594 [cs.CL] https://arxiv.org/abs/2412.03594

[47] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC] https://arxiv.org/abs/2401.09670

A Algorithms

Algorithm 2 ComputeHashes(T, K)

Require: Token sequence $T = (t_1, \ldots, t_N)$; chunk size K
Ensure: Cumulative prefix-hash vector $H = (h_1, \ldots, h_C)$

$H \leftarrow []$; $C \leftarrow \lceil N/K \rceil$
Initialise incremental xxHash-64 state s
for $i \leftarrow 1$ **to** N **do**
  Feed t_i into s
  if $i \bmod K = 0$ **or** $i = N$ **then**
    H.Append(Digest(s))
  end if
end for
return H

Algorithm 3 Insert(r, T)

Require: Request r; token sequence T; working set *working_set*; reverse index *waiting*; min-heap *waiting_heap*
Ensure: r registered and ready for scheduling

$H_r \leftarrow$ ComputeHashes(T, K)
request_hashes$[r] \leftarrow H_r$
$m_r \leftarrow 0$
for $\ell \leftarrow 1$ **to** $|H_r|$ **do**
  $h \leftarrow h^r_\ell$
  if $(\ell, h) \notin$ *working_set* **then**
    $m_r \leftarrow m_r + 1$
  end if
  waiting$[h] \leftarrow$ *waiting*$[h] \cup \{r\}$
end for
miss$[r] \leftarrow m_r$
waiting_heap.Push$(m_r,\ r)$

Algorithm 4 AddToBatch(r)

Require: H_r; ref; *working_set*; *waiting*; *miss*; *waiting_heap*; tip (ℓ^*, h^*); *active*
Ensure: Updated ref, *working_set*, *miss*, (ℓ^*, h^*)

active $\leftarrow$ *active* $\cup \{r\}$; *waiting_requests* $\leftarrow$ *waiting_requests* $\setminus \{r\}$
if $|$*active*$| = 1$ **then**
  $(\ell^*, h^*) \leftarrow (|H_r|,\ h^r_{|H_r|})$; **return**
end if
$\ell_{\text{tip}} \leftarrow \min(\ell^*,\ |H_r|)$; $(\ell^*, h^*) \leftarrow (0, \perp)$
for $\ell \leftarrow |H_r|$ **downto** 1 **do**
  if ref$[h^r_\ell] = 0$ **then**
    working_set $\leftarrow$ *working_set* $\cup \{(\ell, h^r_\ell)\}$
    for $w \in$ *waiting*$[h^r_\ell],\ w \notin$ *active* **do**
      miss$[w]$ $-= 1$; *waiting_heap*.Push(*miss*$[w], w$)
    end for
  end if
  ref$[h^r_\ell]$ += 1
  if $\ell^* = 0$ **and** $\ell \le \ell_{\text{tip}}$ **and** $(\ell, h^r_\ell) \in$ *working_set* **then**
    $(\ell^*, h^*) \leftarrow (\ell,\ h^r_\ell)$ ▷ LCA found; continue loop for remaining working set updates
  end if
end for

Algorithm 5 FINISH(r)

Require: H_r; ref; *working_set*; *waiting*; *miss*; *waiting_heap*; tip (ℓ^*, h^*); *active*
Ensure: Updated ref, *working_set*, *miss*, (ℓ^*, h^*)

```
active ← active \ {r}
for ℓ ← 1 to |H_r| do
    ref[h_ℓ^r] −= 1
    if ref[h_ℓ^r] = 0 then
        working_set ← working_set \ {(ℓ, h_ℓ^r)}
        for w ∈ waiting[h_ℓ^r], w ∉ active do
            miss[w] += 1; waiting_heap.PUSH(miss[w], w)
        end for
    end if
end for
if active = ∅ then
    (ℓ*, h*) ← (0, ⊥); return
end if
if |active| = 1 then
    let q be the sole element of active
    (ℓ*, h*) ← (|H_q|, h^q_{|H_q|}); return
end if
while ℓ* + 1 ≤ min_{q∈active} |H_q| do    ▷ try to extend tip forward
    ℓ ← ℓ* + 1
    if ref[h_ℓ^{q_0}] = |active| for arbitrary q_0 ∈ active then    ▷ all active requests agree at level ℓ
        (ℓ*, h*) ← (ℓ, h_ℓ^{q_0})
    else
        break
    end if
end while
REMOVE(r)    ▷ cleans reverse index and request hashes
```

Algorithm 6 FINDBEST()

Require: *waiting_heap*; *miss*; *active*; tip (ℓ^*, h^*); *working_set*; *request_hashes*
Ensure: $(r^*, \ell^*_{before}, \ell^*_{after}, peers)$ or $\perp$

```
if waiting_requests = ∅ then return ⊥
end if
if cache_valid then return cached_result
end if
while waiting_heap ≠ ∅ do
    (m, r) ← waiting_heap.POP()
    if r ∈ active or r ∉ miss or miss[r] ≠ m then continue
    end if
    r* ← r;  waiting_heap.PUSH(m, r*)    ▷ restore for future calls
    ℓ*_before ← ℓ*
    ℓ*_after ← ℓ where ℓ is the largest ℓ ≤ min(ℓ*, |H_{r*}|) s.t. (ℓ, h_ℓ^{r*}) ∈ working_set
    peers ← |{w ∈ waiting_requests : h^w_{ℓ*_after} = h^{r*}_{ℓ*_after}}|
    Cache and return (r*, ℓ*_before, ℓ*_after, peers)
end while
return ⊥
```

B Throughput vs Batch Size for small requests with no prefix shared

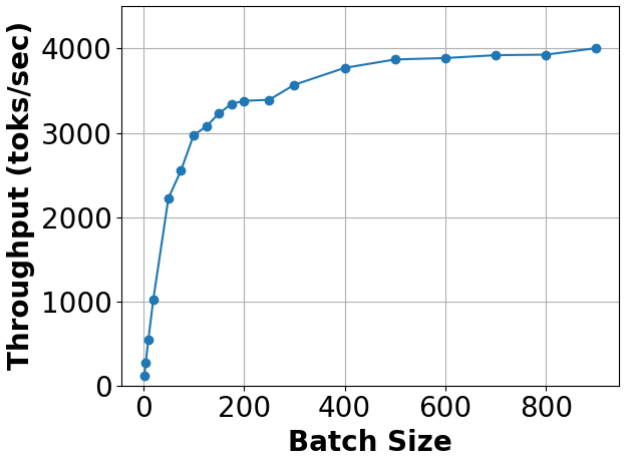


Figure 39. Throughput vs Batch Size

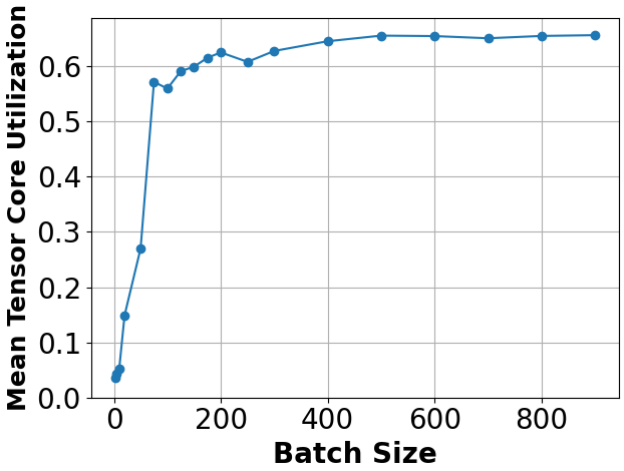


Figure 40. Tensor Core Utilization vs Batch Size

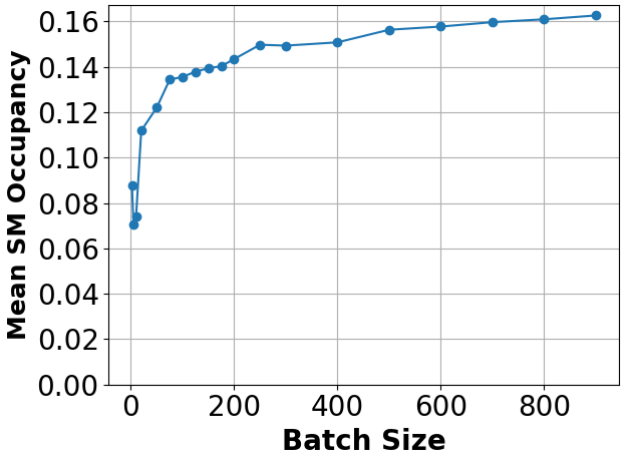


Figure 41. Mean SM Occupancy vs Batch Size

C Impact of Sharing Position on Decode Throughput - $5L$

To further strengthen our observations, we also present results for a sequence length of $5L$ in Figure 42, where we extend the length of the top shared nodes by L for all of these settings (the no sharing patterns remain unchanged, with both prefix groups of length $5L$ each). plot in appendix, but do we need this here?

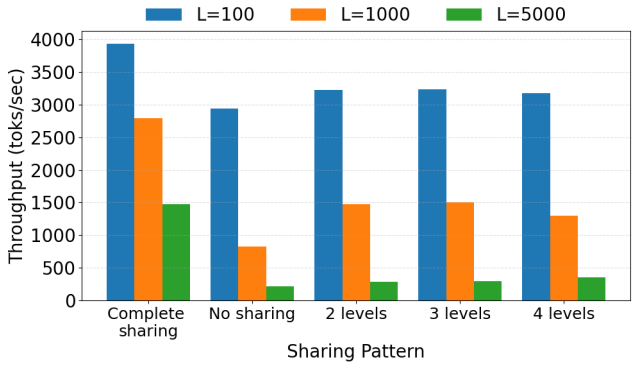


Figure 42. Throughput vs sharing patterns

D Homo vs Hetero - No Warmup - Varying Decode Tokens

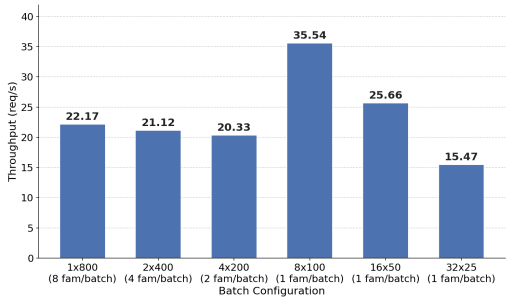


Figure 43. with warmup - 50 decode tokens

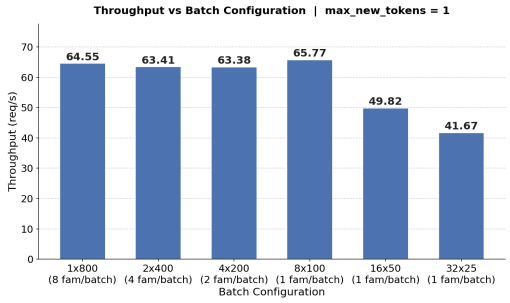


Figure 44. Throughput vs Batch Configuration

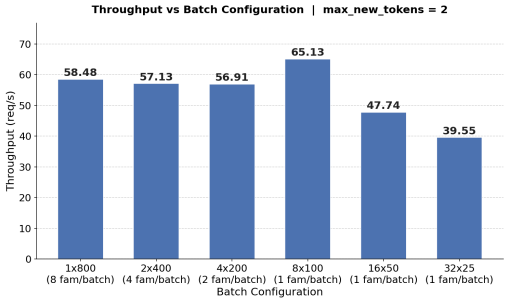


Figure 45. Throughput vs Batch Configuration

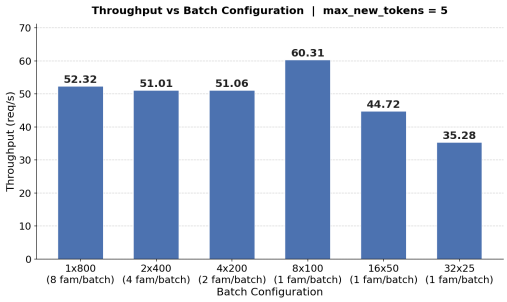


Figure 46. Throughput vs Batch Configuration

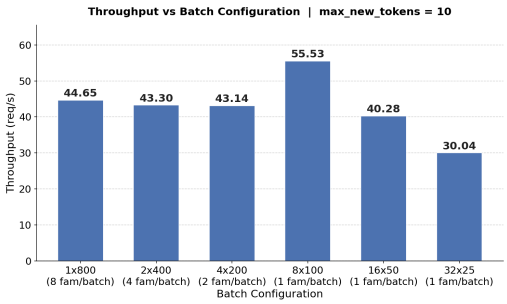


Figure 47. Throughput vs Batch Configuration

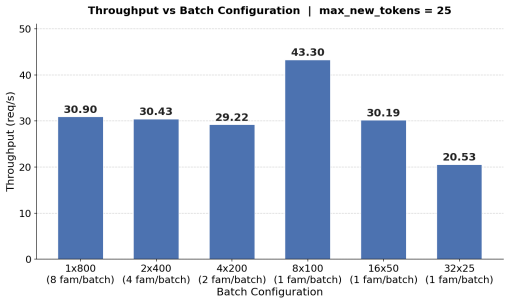


Figure 48. Throughput vs Batch Configuration

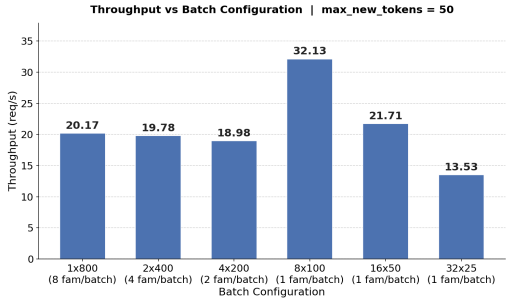


Figure 49. Throughput vs Batch Configuration

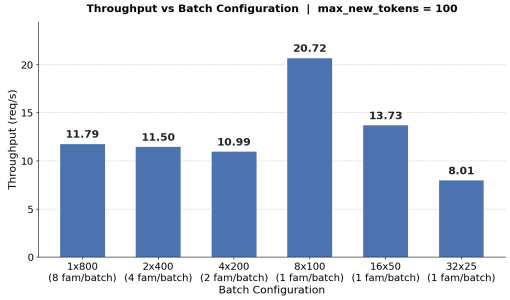


Figure 50. Throughput vs Batch Configuration

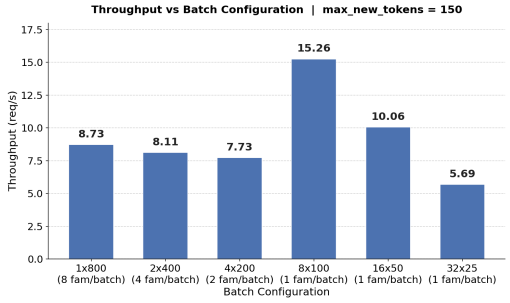


Figure 51. Throughput vs Batch Configuration

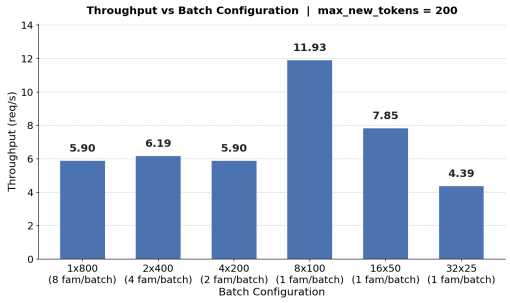


Figure 52. Throughput vs Batch Configuration

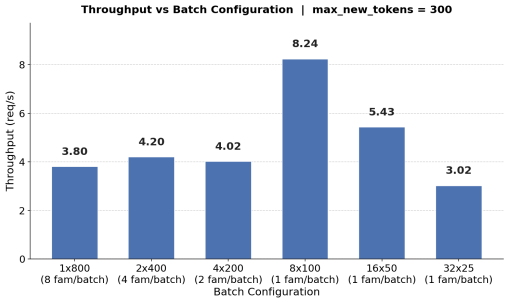


Figure 53. Throughput vs Batch Configuration

E Mean SM Occupancy vs Batch Size

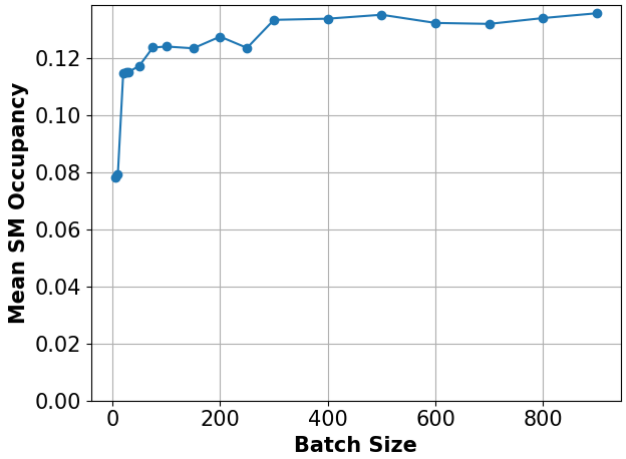


Figure 54. SM Occupancy

F Cache Working Set Size Theory

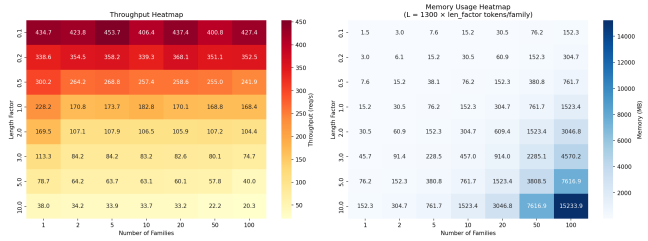


Figure 55. QWEN Throughput and KV Cache Memory across various sequence lengths and number of families